

Research paper

Towards reliable uncertainty quantification via deep ensemble in multi-output regression task[☆]Sunwoong Yang, Kwanjung Yee^{*}

Department of Aerospace Engineering, Seoul National University, Seoul, 08826, Republic of Korea

ARTICLE INFO

Keywords:

Regression task
Predictive uncertainty
Deep ensemble
Uncertainty calibration
Bayesian optimization

ABSTRACT

This study aims to comprehensively investigate the deep ensemble approach, an approximate Bayesian inference, in the multi-output regression task for predicting the aerodynamic performance of a missile configuration. To this end, the effect of the number of neural networks used in the ensemble, which has been blindly adopted in previous studies, is scrutinized. As a result, an obvious trend towards underestimation of uncertainty as it increases is observed for the first time, and in this context, we propose the deep ensemble framework that applies the post-hoc calibration method to improve its uncertainty quantification performance. It is compared with Gaussian process regression and is shown to have superior performance in terms of regression accuracy ($\uparrow 55\sim 56\%$), reliability of estimated uncertainty ($\uparrow 38\sim 77\%$), and training efficiency ($\uparrow 78\%$). Finally, the potential impact of the suggested framework on the Bayesian optimization is briefly examined, indicating that deep ensemble without calibration may lead to unintended exploratory behavior. This UQ framework can be seamlessly applied and extended to any regression task, as no special assumptions have been made for the specific problem used in this study.

1. Introduction

We are entering an era of high-performance computing technologies and they have enabled engineers to efficiently obtain vast amounts of data, so-called big data. Accordingly, numerous data-driven approaches have been studied to derive physical insights from the growing number of available datasets. The most popular but most fundamental one is to utilize a given dataset to train a regression model (also referred to as a surrogate model), which is used to predict quantities of interest (QoIs) (Jeong et al., 2005; Nikolopoulos et al., 2022; Yang et al., 2022; Hong et al., 2023). This straightforward approach can be leveraged for a variety of applications, from exploration during the design optimization process (Yang and Yee, 2022b) to the prediction of high-dimensional data via reduced-order modeling (Kang et al., 2022). Furthermore, from the perspective that the regression model can accelerate the realization of digital twins by replacing the high-demand simulations required within its procedure (Yang et al., 2024; VanDerHorn and Mahadevan, 2021), its potential seems boundless.

However, such impacts cannot be fully achieved by the regression model alone. In real-world engineering problems, *knowing what it does not know and therefore improving interpretability* is an indispensable issue. In the decision-making process based on the regression model,

engineers should consider the predictive uncertainty derived from insufficient train data and imperfect regression model (Zhang et al., 2022). Otherwise, blind faith in regression models, especially during risk assessment and management procedures, can lead to unexpected and therefore disastrous outcomes. The most common approach to deal with this issue is to perform Bayesian optimization, also known as efficient global optimization in engineering fields (Jones et al., 1998; Yang and Yee, 2022b; Chae et al., 2010; Kanazaki et al., 2007). Briefly, it aims to reduce model uncertainty by iteratively updating the model based on the acquisition function (Snoek et al., 2012; Shin et al., 2020; Shimoyama et al., 2013), which contains uncertainty information (Fig. 1). Since the Bayesian optimization process requires uncertainty quantification (UQ), whether the model quantifies the uncertainty over its prediction is the key consideration for engineers in determining which regression model to utilize.

Gaussian process regression (GPR)—also known as Kriging—is one of the most widely used regression models capable of UQ in various engineering fields (Rhode, 2020; Wang et al., 2022; Yang et al., 2020; Quirante et al., 2018; Quirante and Caballero, 2016; Keßler et al., 2019; Zhong et al., 2019; Yang and Yee, 2022a; Sugimura et al., 2009; Park et al., 2022; Muñoz et al., 2023; Yıldız, 2020, 2022). GPR allows engineers to identify which predictions are unreliable by providing

[☆] This work was presented at the AIAA SciTech 2023 Forum, 23–27 January 2023, National Harbor, MD & Online (Yang and Yee, 2023).

^{*} Corresponding author.

E-mail addresses: sunwoongy@kaist.ac.kr (S. Yang), kjyee@snu.ac.kr (K. Yee).

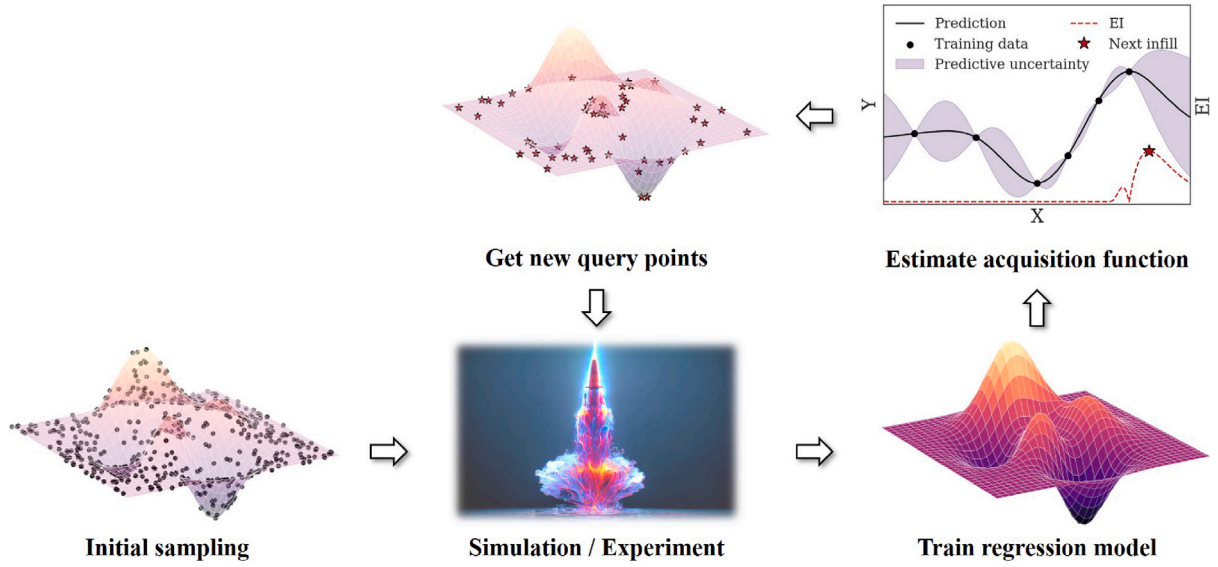


Fig. 1. Flowchart of Bayesian optimization.

predictive uncertainty, and it has become the most prevalent regression model for Bayesian optimization (Jeong et al., 2005; Yang and Yee, 2022b; Namura et al., 2016; Shimoyama et al., 2013). However, GPR is notorious for its time complexity of $O(n^3)$ and memory complexity of $O(n^2)$, where n denotes the dataset size (Cheng and Boots, 2017; Wang et al., 2017). Even in multi-output regression tasks, since a GPR is trained for each output independently, the required training time increases linearly with respect to the output dimension, and the correlations within outputs become completely ignored (Lin et al., 2021, 2022; Wang and Chen, 2015).

In this regard, Bayesian neural networks (BNNs) (MacKay, 1992, 1995; Fernández et al., 2023) can be effective alternatives for the following reasons: (1) their universal approximation capability (Hornik et al., 1989; Barron, 1993); (2) scalability to large datasets due to mini-batch training (Meng et al., 2021); and (3) multi-output prediction only with a single regression model. Since BNNs aim to learn the probability distributions of the model parameters on the basis of Bayesian inference, they can estimate the uncertainty of their prediction, whereas traditional neural networks (NNs) only provide point estimates. However, their additional model parameters lead to slower convergence during the training (Zhang et al., 2019) and require significant modifications to the conventional framework of NNs, leading to cumbersome and knotty training algorithms (Gal et al., 2016; Lakshminarayanan et al., 2017; Fernández et al., 2022). Such computational complexity and inefficiency prevent BNNs from being a viable option for engineers who prioritize practicality and are not familiar with Bayesian formalism.

Recently, easy-to-use but scalable approaches for approximating Bayesian inference have attracted the attention of engineers. Especially, deep ensemble (DE) (Lakshminarayanan et al., 2017; Yang and Yee, 2023) and MC-dropout (Gal and Ghahramani, 2016; Deruyttere et al., 2021) require only a few modifications to standard (or vanilla) NNs, demonstrating their applicability to the fields of engineering. However, since MC-dropout has controversial issues about whether or not it is Bayesian inference (Osband, 2016; Hron et al., 2017, 2018; Folgoc et al., 2021), it is out of our focus; see Appendix A. DE, an approach to quantify the predictive uncertainty by leveraging ensembles of NNs, was first proposed by Lakshminarayanan et al. (2017). Their idea is so “simple and straightforward” that it only requires training multiple NNs in parallel on the same training dataset. Despite its simplicity, several researchers have recognized that the DE provides not only accurate predictions, but also robust, reliable, and practically useful uncertainty on a wide variety of architectures and datasets, even on

out-of-distribution (OOD) examples (Gustafsson et al., 2020; Fort et al., 2019; Ovadia et al., 2019; Ashukha et al., 2020). Finally, it has come to be treated as the “gold standard for accurate and well-calibrated predictive distributions” (Wilson and Izmailov, 2020).

However, most previous studies have focused on verifying whether DE accurately estimates the uncertainty in classification tasks (Lakshminarayanan et al., 2017; Rahaman et al., 2021; Wu and Gales, 2021; Fort et al., 2019; Ovadia et al., 2019; Ashukha et al., 2020). Its comprehensive validation has not been conducted in multi-output regression tasks, which are the most common problems in practical engineering disciplines. For example, de Becdelievre and Kroo (2021) and Pawar et al. (2022) utilized DE for tailless aircraft range optimization and boundary layer flow prediction tasks, respectively, without any validation of the estimated uncertainty in their problems. In this sense, our research focuses on a thorough validation of the DE approach in multi-output regression tasks, while comparing it with GPR, both in terms of regression accuracy and reliability of the estimated uncertainty. Especially, we seek to overcome the limitations of existing studies that blindly adopted the number of NNs used in DE without sufficient explanation of their effects (Pocvičiūtė et al., 2022; Ovadia et al., 2019; Ilg et al., 2018; de Becdelievre and Kroo, 2021; Linmans et al., 2020; Rahaman et al., 2021; de Becdelievre and Kroo, 2021; Egele et al., 2022; Maulik et al., 2023). Finally, a tendency of the quantified uncertainty to become underconfident with the number of NNs is observed and a practical calibration method is proposed to be applied. The corresponding effects are verified quantitatively with two uncertainty evaluation criteria, and their potential impact on Bayesian optimization is briefly investigated. The main contributions of this work can be summarized as follows:

1. First attempt to validate DE approach in the multi-output regression task.
2. The effect of the number of NNs used for DE is comprehensively investigated and two different criteria are utilized for rigorous validation of its uncertainty quality.
3. Accordingly, an increasing trend of underconfidence with the increasing number of NNs is first empirically observed in the regression task, and its analytical explanation is derived.
4. A simple post-hoc calibration method is applied to DE models for the correction of unsatisfactory uncertainty quality and its effectiveness is verified both qualitatively and quantitatively.
5. The potential impact of the proposed calibration method on Bayesian optimization is briefly examined: the possibility that

different estimates of uncertainty could lead to different exploration behavior is examined.

6. Throughout the above procedures, GPR – the most well-known UQ model – is compared with DE, and the effectiveness of DE over GPR is confirmed.

The rest of this paper is organized as follows. In Section 2, the background on how to implement DE and evaluate its uncertainty quality is described. In Section 3, the application of DE to a multi-output regression task in aerospace engineering is elaborated. It provides a thorough validation of DE models compared to GPR models, both in terms of prediction accuracy and uncertainty quality. In Section 4, a simple post-hoc calibration method is applied and its effects on uncertainty quality and Bayesian optimization are investigated. Finally, in Section 5, the conclusion and future work of this study are presented.

2. Implementation and evaluation of DE

DE was first proposed by Lakshminarayanan et al. (2017) for the simple and scalable estimation of predictive uncertainty. Although its idea can be seen as a straightforward extension of NNs (making use of multiple NNs), DE has received little attention in the engineering disciplines, in contrast to its reputation in computer science. This is due to the lack of previous works explaining its algorithm friendly and comprehensively, and therefore the purpose of this section is to fill the academic gap by elaborating on the DE methodology and its validation. First, we briefly introduce the NNs (Section 2.1) before moving on to DE. Then, the background of how to implement DE (Section 2.2) and how to evaluate its uncertainty quality (Section 2.3) is described.

2.1. Neural networks (NNs)

Engineers from various disciplines have been drawn to NNs due to their universal approximation capability (Hornik et al., 1989; Barron, 1993), ability to scale to large datasets through mini-batch training (Meng et al., 2021), and capability of making multi-output predictions with a single regression model. This section provides a brief theoretical overview of these NNs.

The feed-forward mechanism propagates the data obtained from the input layer of NNs to the output layer. In this procedure, information moves via an affine transformation as follows:

$$y = Wx + b, \quad (1)$$

where x is a vector of nodes in the input layer and y is that in the output layer. W and b are the weight matrix and bias vector between the input and output layers, respectively. Regardless of the number of hidden layers between the input and output layers, nonlinearity between x and y cannot be captured since they are linearly correlated in Eq. (1). In this context, the concept of an activation function that modifies the output of NNs is introduced. By incorporating nonlinear activation functions at each layer, NNs can perform nonlinear modeling. A variety of activation functions are available, including the LeakyReLU function (Maas et al., 2013), which is as follows:

$$f(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{otherwise} \end{cases} \quad (2)$$

where a stands for a non-zero small gradient (0.01 for this study). And this activation function (Eq. (2)) is applied to the output of the previous layer. The correspondingly transformed output is then utilized as the input for the subsequent layer. This process, known as feed-forward, is repeated through the hidden layers.

However, the feed-forward itself cannot achieve the expected accuracy because it lacks an algorithm for training the parameters of NNs, namely weights (W) and biases (b). To address this issue, the backpropagation training algorithm was introduced, which minimizes the loss function by adjusting the parameters to make the predicted

values of the NNs similar to the desired target values as the training progresses (Rumelhart et al., 1986). To this end, gradient descent optimization techniques, such as Adagrad (Duchi et al., 2011), RM-Sprop (Tieleman et al., 2012), and Adam (Kingma and Ba, 2014), are utilized to minimize the loss function. In particular, Adam has become increasingly popular due to its strengths in dealing with sparse gradients and non-stationary objectives, combining Adagrad and RM-Sprop (Kingma and Ba, 2014). As the feed-forward process and back-propagation with gradient descent are repeated iteratively, the loss function decreases to the desired level so that the training stops. The converged weights and biases of the NN model can then be used to make almost real-time predictions using the feed-forward operation. Only the essential aspects of NNs are presented here, as many studies have already described them. More information on NNs can be found in Goodfellow et al. (2016).

2.2. Deep ensemble (DE)

The NNs discussed above are often considered “overconfident” because they do not provide any measure of uncertainty. For those who are interested in UQ, DE can be an alternative approach. DE is based on an ensemble of NNs, but there is a key distinction: unlike a standard NN, which only outputs QoIs as $\mu(x)$, the NN used for DE outputs them as a Gaussian distribution, $N(\mu(x), \sigma^2(x))$. That is, it assumes that QoIs are sampled from $N(\mu(x), \sigma^2(x))$ and aims to provide information about this distribution by outputting $\mu(x)$ and $\sigma^2(x)$. Here, $\mu(x)$ refers to the estimated/predicted value and $\sigma^2(x)$ refers to the estimated/predicted variance. It should be noted that the estimated variance $\sigma^2(x)$ indicates the aleatory uncertainty (uncertainty arising from noise inherent in the training data) regarding the estimated value $\mu(x)$ (Solopchuk and Zénon, 2021; Laves et al., 2021). With this specific NN architecture, the number of final nodes is doubled since it outputs not only the standard outputs, $\mu(x)$, but also the uncertainty about them, $\sigma^2(x)$. Due to the probabilistic distribution it provides, this type of NN is referred to as a probabilistic NN.

The probabilistic NN architecture is adopted in the DE model since the vanilla NN structure cannot apply the proper scoring rule, which is the criterion for estimating the quality of predictive uncertainty (Gneiting and Raftery, 2007). Lakshminarayanan et al. (2017) emphasized that with the vanilla NN architecture, which provides only the estimated value $\mu(x)$, the mean squared error (MSE) would be used as the loss function:

$$\text{MSE} = (y - \mu(x))^2 \quad (3)$$

and therefore the information about the predictive uncertainty is entirely disregarded during the training. To address this issue, they proposed utilizing a probabilistic NN that can output both $\mu(x)$ and $\sigma^2(x)$. It allows the use of the proper scoring rule, negative log-likelihood (NLL), which is the standard metric for assessing the quality of probabilistic models (Hastie et al., 2009):

$$\text{NLL}(\mu(x), \sigma^2(x), y) = -\log(p_\theta(y|x)) = -\frac{\log \sigma^2(x)}{2} - \frac{(y - \mu(x))^2}{2\sigma^2(x)} - \frac{\log 2\pi}{2} \quad (4)$$

This NLL allows the intuitive interpretations as follows (Kendall and Gal, 2017; Guo et al., 2017). (1) When some training points have high MSE, $(y - \mu(x))^2$, the impact of the term $\frac{(y - \mu(x))^2}{2\sigma^2(x)}$ is relatively significant compared to $\frac{\log \sigma^2(x)}{2}$. Therefore, the model is trained to output high denominator value, $\sigma^2(x)$, at the corresponding points to reduce the NLL. (2) At training points with low MSE, the term $\frac{\log \sigma^2(x)}{2}$ becomes relatively dominant and thus the model is encouraged to output low $\sigma^2(x)$ at those points. In summary, the NLL scoring rule-based training algorithm for the probabilistic NN facilitates the learning of reliable predictive uncertainty by estimating high uncertainty where prediction error is high and low uncertainty where prediction error is low. It

should be noted that this cannot be accomplished in vanilla NN with MSE loss function.

However, using a single probabilistic NN is limited to estimating the aleatory uncertainty. To estimate the epistemic uncertainty arising from the model parameters due to insufficient training data, a further step is required. Lakshminarayanan et al. (2017) suggested the use of multiple probabilistic NNs, called deep ensemble (DE), to quantify both aleatory and epistemic uncertainties. Specifically, they aimed to capture the epistemic uncertainty by using the multiple probabilistic NNs trained on the identical dataset (also identical architectures for NNs are used). The overall training procedure is summarized in Algorithm 1. There are two notable points herein: (1) the random initialization of the model parameters of the NNs in line 2; and (2) the random shuffling of the training dataset due to mini-batches in line 5. These two factors are regarded as the main causes of the individual NN with identical architecture in the ensemble being able to be trained with enough diversity (Lakshminarayanan et al., 2017). See Fort et al. (2019) for further information, which examined the effects of random initialization and random shuffling.

Algorithm 1 Training procedure of DE

```

1: Split the train dataset  $X$  (with input  $x$  and output  $y$ ) into  $J$ 
   mini-batches.
2: Randomly initializes model parameters of the  $M$  probabilistic NNs
   and set training epochs.
3: for  $i = 1 : M$  do                                ▷ Loop for NN (parallelizable)
4:   for epochs do                                    ▷ Loop for epoch
5:     for  $j = 1 : J$  do                                ▷ Loop for mini-batch
6:        $\mu_{ij}, \sigma_{ij}^2 = \text{NN}_i(x_{ij})$                 ▷ Feed-forward with mini-batch  $x_j$ 
7:        $\mathcal{L}_{ij} = \text{NLL}(\mu_{ij}, \sigma_{ij}^2, y_{ij})$           ▷ Calculate NLL
8:        $\theta_i = \theta_i - \text{learning rate} * \delta \mathcal{L}_{ij} / \delta \theta$     ▷ Update  $\text{NN}_i$ 
9:     end for
10:   end for
11: end for

```

To see how the ensemble of probabilistic NNs trained in Algorithm 1 estimates two types of uncertainty, let $\mu_i(x)$ and $\sigma_i^2(x)$ be the predictive mean and predictive variance output by the i th individual NN. Herein, the predicted probabilities of y from the i th NN can be expressed as $N(\mu_i(x), \sigma_i^2(x))$, indicating that there are multiple Gaussian distributions according to each NN in the ensemble. Lakshminarayanan et al. (2017) suggested approximating the final probability of the output as a mixture of Gaussian probabilities as follows:

$$\hat{\mu} = \frac{1}{M} \sum_{i=1}^M \mu_i, \quad (5)$$

$$\underbrace{\hat{\sigma}^2}_{\text{predictive uncertainty}} = \frac{1}{M} \sum_{i=1}^M \sigma_i^2 + \left(\frac{1}{M} \sum_{i=1}^M \mu_i^2 - \hat{\mu}^2 \right) \quad (6)$$

$$= \underbrace{E(\sigma_i)}_{\text{aleatory uncertainty}} + \underbrace{Var(\mu_i)}_{\text{epistemic uncertainty}}$$

where M is the number of probabilistic NNs used for the ensemble. Accordingly, the final predictive value of DE is $\hat{\mu}$ and the final predictive uncertainty is $\hat{\sigma}^2$. As in Eq. (6), the predictive uncertainty can be decomposed into aleatory and epistemic uncertainty; see Scalia et al. (2020) and Hu et al. (2021) for more details. It should be noted that no additional training algorithm is required after the training of probabilistic NNs in Algorithm 1: only the mixture process of already trained NNs in Eqs. (5) and (6) is required. The overall flowchart of DE from the training of probabilistic NNs to the final prediction is schematically shown in Fig. 2.

2.3. Uncertainty quality evaluation

In the previous Section 2.2, we explored the ability of the DE technique to determine predictive uncertainty. However, engineers who are interested in predicting uncertainty require more than just the feasibility of UQ; they also require confidence in the reliability of the estimated uncertainty. Unfortunately, previous studies that employed GPR to evaluate predictive uncertainty in engineering fields have disregarded this point. Consequently, the purpose of this section is to address this gap by presenting two criteria for assessing the accuracy/reliability of estimated predictive uncertainty. These techniques are applicable to any regression model performing UQ, such as GPR and DE.

2.3.1. AUCE

The most widely used metric to evaluate the reliability of uncertainty is the area under the calibration error curve (AUCE) (Kuleshov et al., 2018; Gustafsson et al., 2020). The primary goal of this measure is to ensure that the confidence intervals (CI) estimated by the model are accurate in practice. The concept of AUCE is shown schematically in Fig. 3. In Fig. 3(a), the CI labeled “Well-calibrated 60% CI” contains 60% of the test dataset (6 out of 10 points), where test dataset indicates the dataset used to verify the quality of the estimated uncertainty. Thus, a well-calibrated model would have a 60% CI that actually contains 60% of the test data. On the other hand, if the 60% CI contains more than 60% of the dataset (8 out of 10 points), the model is considered underconfident, which corresponds to the case of “Underconfident 60% CI”. This means that the model is not confident enough about its prediction and overestimates its CI. Conversely, if the 60% CI contains less than 60% of the dataset (4 out of 10 points, “Overconfident 60% CI” case), the model is considered overconfident, meaning that it is too confident in its prediction and thus estimates a narrower CI than it actually should.

The difference between the CI estimated by the model and the actual data it contains can be assessed visually by the CI-based reliability plot shown in Fig. 3(b). This plot compares the predicted CI from the model on the x-axis with the observed CI measured with the test dataset on the y-axis. To clarify, consider the situation depicted in Fig. 3(a). In the underconfident case, which corresponds to point P1 ($x = 0.6$, $y = 0.8$), the predicted 60% CI actually corresponds to the observed 80% CI because 8 out of 10 points are included. Point P2 represents the well-calibrated case, where the predicted 60% CI by the model matches the actual 60% of data contained in the CI. In contrast, point P3 represents the overconfident case, where the model predicts a 60% CI that actually contains only 40% of the data. In this context, the line $y = x$ represents an ideally well-calibrated model where the predicted CI perfectly matches the observed CI. The algorithm for the CI-based reliability plot is summarized in Algorithm 2.

By utilizing this CI-based reliability plot, the AUCE, which is a metric that evaluates the quality of the estimated uncertainty, can be derived. In detail, it is calculated as the area between the ideal line $y = x$ and the reliability plot of the model. The hatched area in Fig. 3(b) corresponds to the AUCE of the underconfident model, and the mathematical expression for the AUCE is provided in the following equation (Gustafsson et al., 2020):

$$\text{AUCE} = \frac{1}{K} \sum_{i=1}^K |\hat{p} - p_i| \quad (7)$$

where K refers to the number of CI candidates as in Algorithm 2. By definition, a low AUCE value implies that the predictive uncertainty quantified by the model is reliable (or well-calibrated). Additional information on AUCE can be found in Naeini et al. (2015), Gustafsson et al. (2020), and Scalia et al. (2020).

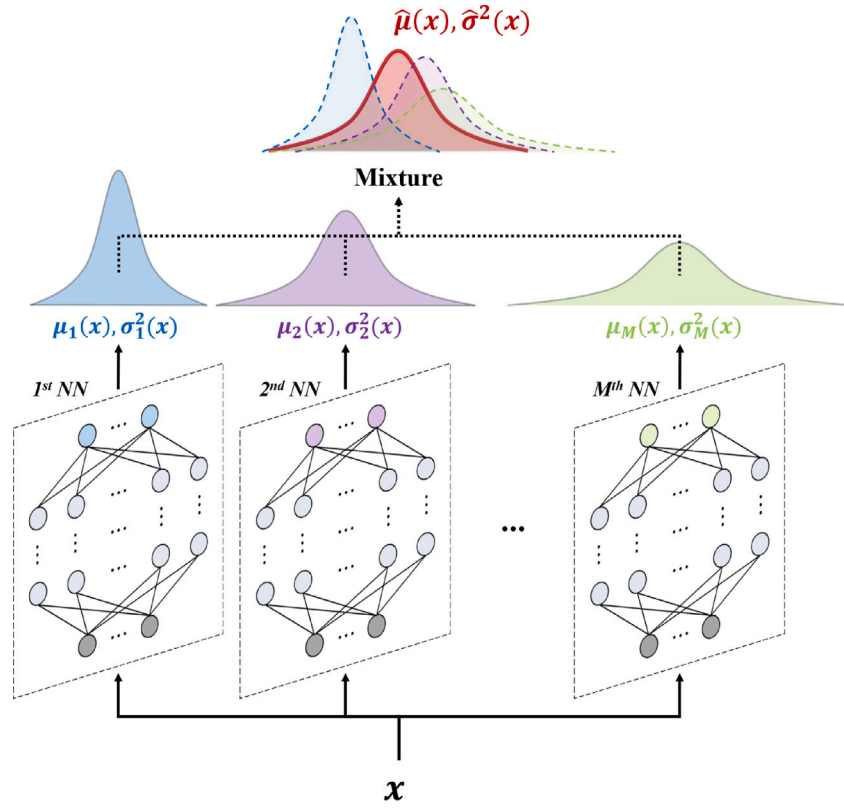


Fig. 2. Flowchart of DE approach.

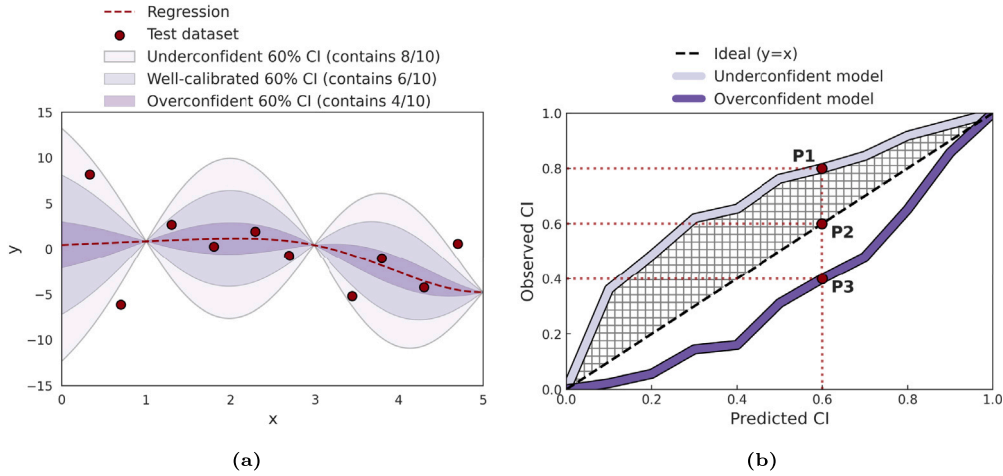


Fig. 3. (a) Illustration of well-calibrated/miscalibrated models: 60% CI of the well-calibrated model contains 60% of the test data, whereas that of the underconfident and overconfident model contains 80% and 40% of the data, respectively. (b) Illustration of CI-based reliability plot.

2.3.2. ENCE

Despite its reputation as a metric of uncertainty quality, AUCE has a critical shortcoming in that it only considers the average over the entire test dataset rather than individuals as mentioned by Levi et al. (2022). Moreover, they analytically and empirically elaborated that AUCE can be zero even when the predicted distribution is statistically independent from that of the ground truth. In this context, they proposed a novel approach to evaluate the quality of uncertainty, the expected normalized calibration error (ENCE). It was first proposed based on the intuitive assumption: for the well-calibrated model, the estimated uncertainty $\sigma^2(x)$ will be equal to $(y - \mu(x))^2$, MSE. This condition can be expressed mathematically as follows, implying that a higher estimated variance should correspond to a higher expected

MSE (Phan et al., 2018):

$$\mathbb{E}_{x,y}[(y - \mu(x))^2 | \sigma^2(x)] = \sigma^2(x) \quad (8)$$

The above Eq. (8) indicates that the ideally (perfectly) well-calibrated model will have an expected error exactly equal to predictive uncertainty. In this sense, whether the model is well-calibrated can be visually inspected using the error-based reliability plot (Scalia et al., 2020; Levi et al., 2022): x-axis as root mean squared error (RMSE), $y - \mu(x)$, and y-axis as root of the mean variance (RMV), $\sigma(x)$. Fig. 4 illustrates it, and by its definition in Eq. (8), $y = x$ line indicates the ideally calibrated model. The procedure for its plotting is summarized in Algorithm 3.

Algorithm 2 Procedure for CI-based reliability plot

```

1: Prepare the test dataset  $X$  (with input  $x$  and output  $y$ ).
2: Define candidates of CI to be investigated:  $P = \{p_1, p_2, \dots, p_K\}$ .
3:  $D = \emptyset$   $\triangleright$  Initialize dataset  $D$  to be plotted as y-axis
4: for  $i = 1 : K$  do  $\triangleright$  Loop for  $P$ 
5:    $count = 0$   $\triangleright$  Initialize  $count$ 
6:   Find  $Q(\frac{p_i + 1}{2} | \mu, \sigma^2)$ , which is  $\frac{p_i + 1}{2}$  quantile of  $N(\mu, \sigma^2)$ .
7:   for  $j = 1 : \text{length}(X)$  do  $\triangleright$  Loop for  $X$ 
8:     if  $-Q(\frac{p_i + 1}{2} | \mu(x_j), \sigma^2(x_j)) \leq y_j \leq Q(\frac{p_i + 1}{2} | \mu(x_j), \sigma^2(x_j))$ 
9:        $count += 1$   $\triangleright$  If test data is within the estimated CI
10:    end if
11:  end for
12:   $\hat{p} = count / \text{length}(X)$   $\triangleright$  Calculate observed CI
13:   $D = D \cup \hat{p}$   $\triangleright$  Append  $\hat{p}$  to  $D$ 
14: end for
15: Plot CI-based reliability plot: x-axis with  $P$  and y-axis with  $D$ .

```

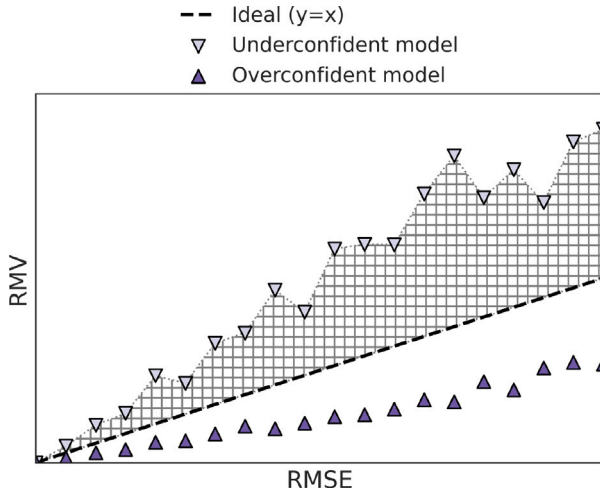


Fig. 4. Illustration of error-based reliability plot. Underconfident model overestimates RMV relative to RMSE, while overconfident model underestimates RMV. The ideal model estimates the equivalent RMV and RMSE as the $y = x$ black dashed line.

Algorithm 3 Procedure for error-based reliability plot

```

1: Prepare the test dataset  $X$  (with input  $x$  and output  $y$ ).
2: Sort  $X$  according to  $y$  values.
3: Define the number of bins:  $B$  (assume  $B$  divides  $\text{length}(X)$ ).
4: Divide sorted  $X$  into  $B$  bins,  $\tilde{X} = \{\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_B\}$ , such that each  $\tilde{X}_i$  has the same size of  $\text{length}(X)/B$ .
5:  $D_{RMSE} = \emptyset$   $\triangleright$  Initialize dataset  $D_{RMSE}$  to be plotted as x-axis
6:  $D_{RMV} = \emptyset$   $\triangleright$  Initialize dataset  $D_{RMV}$  to be plotted as y-axis
7: for  $i = 1 : B$  do  $\triangleright$  Loop for  $\tilde{X}$ 
8:    $D_{RMSE} = D_{RMSE} \cup \sqrt{\frac{1}{|\tilde{X}_i|} \sum_{x \in \tilde{X}_i} (y(x) - \mu(x))^2}$   $\triangleright$  Append RMSE
9:    $D_{RMV} = D_{RMV} \cup \sqrt{\frac{1}{|\tilde{X}_i|} \sum_{x \in \tilde{X}_i} \sigma^2(x)}$   $\triangleright$  Append RMV
10: end for
11: Plot error-based reliability plot: x-axis with  $D_{RMSE}$  and y-axis with  $D_{RMV}$ .

```

Then, the area between the ideal $y = x$ line and the error-based reliability plot can be calculated. The normalized version of this value refers to ENCE, the second uncertainty quality metric, and is as follows:

$$\text{ENCE} = \frac{1}{B} \sum_{i=1}^B \frac{|\text{RMV}(i) - \text{RMSE}(i)|}{\text{RMV}(i)} \quad (9)$$

where B indicates the number of bins in Algorithm 3. Therefore, the ENCE of the underconfident model in Fig. 4 can be calculated as the hatched area divided by RMV. As with AUCE, the lower the ENCE value, the better the model is calibrated.

2.4. Uncertainty calibration: STD scaling

In situations where the estimated uncertainty from the model is imprecise in terms of AUCE (refer to Section 2.3.1) and ENCE (refer to Section 2.3.2), there are various techniques for calibrating uncertainty. Some of these methods include histogram binning (Zadrozny and Elkan, 2001), isotonic regression (Zadrozny and Elkan, 2002), and temperature scaling (Guo et al., 2017). The first two techniques are non-parametric, and therefore, the number of parameters utilized is dependent on the training dataset size. Conversely, temperature scaling is a parametric approach that needs a fixed number of parameters.

Given the practicality being a crucial consideration in applying UQ techniques to the engineering domain, this research adopts a straightforward approach: temperature scaling. More specifically, the study employs STD (standard deviation) scaling, which is a regression task version of temperature scaling (Levi et al., 2022). With STD scaling, it is only necessary to determine a scalar parameter, denoted as s , which is used to multiply the standard deviation initially estimated by the DE model, $\hat{\sigma}$. The value of s used in the calibration process is selected to minimize the NLL, as shown below:

$$s = \underset{s}{\text{argmin}} \left(\frac{\log(\hat{\sigma}(x))^2}{2} + \frac{(y - \hat{\mu}(x))^2}{2(\hat{\sigma}(x))^2} + \frac{\log 2\pi}{2} \right), \quad (10)$$

Please note that this equation is the simple modification of Eq. (4), where $\sigma(x)$ is replaced by $\hat{\sigma}(x)$. This calibration procedure is completely separate from the training procedure of DE; it is performed after the mixture step in Fig. 2, so it is called the post-hoc or post-process calibration method. It should be emphasized that the model parameters (weights and biases in the NN model) remain unchanged throughout the calibration process. The STD scaling method is intuitively explained as follows: if the estimated uncertainty from the trained model ($\hat{\sigma}(x)$) is poorly calibrated, the calibrated version of the uncertainty $s\hat{\sigma}(x)$ is used in its place. It is important to note that this calibration process is intended solely to correct the estimated uncertainty, and therefore, only the output $\hat{\sigma}(x)$ of the DE changes, while the predictive value $\hat{\mu}(x)$ remains unaltered. The steps involved in the STD calibration process are outlined in Algorithm 4. For calibration, a separate dataset should be used that is distinct from the training and test datasets to ensure calibration generalization (Levi et al., 2022), and therefore, a validation dataset is utilized for the calibration. In multi-output regression tasks, every DE output can be calibrated independently using the number of scaling parameters s equal to the output dimension (this is implemented by the for-loop in line 3 of Algorithm 4). In conclusion, this study uses a straightforward STD calibration method for uncertainty calibration, which involves tuning scalar parameters without modifying trained NNs.

3. Application of DE to aerodynamic performance regression task

This section applies the DE method to a real-world engineering problem of predicting aerodynamic coefficients for a specific missile configuration with varying flow conditions. It aims to validate the performance of DE in multi-output regression tasks since no comprehensive study has been conducted on this topic. The section evaluates both the regression and uncertainty estimation performance of DE and investigates the impact of M , the number of NNs used for the ensemble.

Algorithm 4 STD calibration procedure

- 1: Prepare calibration dataset X (with input x and output y).
- 2: Define candidates of scaling factor: S
- 3: **for** $i = 1 : \text{length}(y)$ **do** ▷ Loop for output dimension of DE
- 4: $s_i = \underset{s \in S}{\operatorname{argmin}} \left(\frac{\log(s\hat{\sigma}_i(x))^2}{2} + \frac{(y_i - \hat{\mu}_i(x))^2}{2(s\hat{\sigma}_i(x))^2} + \frac{\log 2\pi}{2} \right)$
- 5: **end for**
- 6: Utilize s_i to calibrate estimated uncertainty over i th output ▷ Use $s_i\hat{\sigma}_i$ in lieu of $\hat{\sigma}_i$.

3.1. Data preparation and training details

As an engineering problem, the present study adopts the prediction of six aerodynamic coefficients for a particular missile configuration, specifically “Configuration 1” described in the NASA TM-2005-213541 report (Allen, 2005). The Missile Datcom (Blake, 1998) low-fidelity semi-empirical solver is then utilized to compute the coefficients for its configuration, given following five flow conditions: $Ma \in [1.1, 3]$, $\phi \in [-90^\circ, 0^\circ]$ (roll angle), $\delta p \in [-20^\circ, 20^\circ]$ (pitch control fin deflection angle), $\delta r \in [-10^\circ, 0^\circ]$ (roll control fin deflection angle), and $AoA \in [-3^\circ, 23^\circ]$. The resulting aerodynamic coefficients are C_{NF} (normal force coefficient), C_{AF} (axial force coefficient), C_{PM} (pitching moment coefficient), C_{RM} (rolling moment coefficient), C_{YM} (yawing moment coefficient), and C_{SF} (side force coefficient). Subsequently, 9800 points are obtained by full-factorial sampling in the input space and then split into train, validation, and test datasets in the ratio of 8:1:1. The training dataset is utilized to train DE and GPR models, and the validation dataset is used to perform hyperparameter tuning in this section and STD calibration in Section 4.1, and the test dataset is used for regression and UQ performance evaluation in Sections 3.2 and 3.3.

After obtaining the dataset, the next step is to determine the structure of the probabilistic NN to be used for the ensemble: following the work by Lakshminarayanan et al. (2017), the probabilistic NNs with identical architectures are used for ensembling in this study. To this end, grid search is carried out with hyperparameters regarding the network architecture and the diversity between NNs, such as the number of layers, number of nodes, and size of the mini-batch. Other hyperparameters such as the optimizer algorithm, initial learning rate, and total epochs are selected as Adam, 10^{-3} , and 13 000, respectively. The results of the tuning are available in Appendix B.1. Based on NLL and RMSE, a probabilistic NN with 7 hidden layers and 128 nodes is selected, and the mini-batch size is set to 512. Subsequently, different values of the hyperparameter M (2, 4, 8, and 16) are adopted, with each corresponding DE model referred to as DE-2, DE-4, DE-8, and DE-16 in this manuscript. That is, for DE-16, 16 probabilistic NNs with 7 hidden layers and 128 nodes are trained with a mini-batch size of 512, sharing identical hyperparameters with other DE models except M .

GPR models with different kernels are also trained for their hyperparameter tuning. To this end, Matérn 5/2, radial basis function, rational quadratic, and dot-product kernels are examined (Williams and Rasmussen, 2006), and their results also can be found in Appendix B.2. In addition, not only single-output GPR models are tested, but also the multi-output GPR (MOGPR) with radial basis function kernel is trained for more comprehensive comparison with DE (Williams and Rasmussen, 2006; Alvarez et al., 2012; Lin et al., 2021, 2022). Among them, GPR with Matérn 5/2 kernel shows the best performance, and is therefore selected for the comparison with DE throughout this paper. The required training times for DE with selected NN architecture (7 hidden layers, 128 nodes, minibatch size of 512) and GPR (Matérn 5/2 kernel) models using Intel(R) Xeon(R) CPU @ 2.20 GHz are as follows: 10.9 h for GPR, 2.4 h for DE-2, 5 h for DE-4, 9.7 h for DE-8, and 19.4 h for DE-16. Note that GPR requires more training time than DE for both the hyperparameter tuning and final model training.

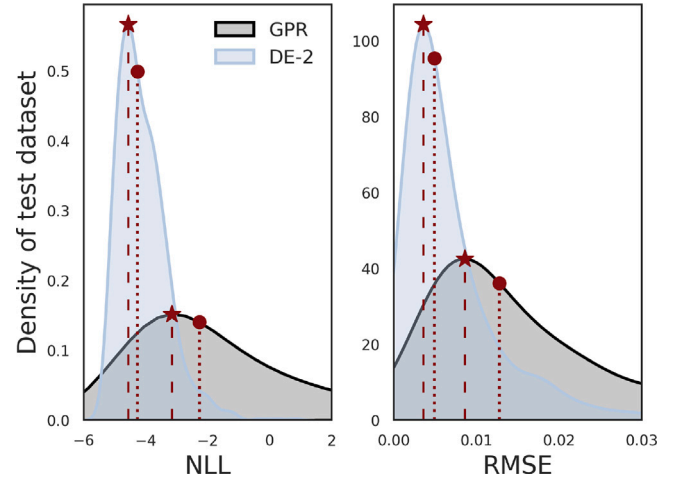


Fig. 5. Comparison of regression accuracy between GPR and DE-2: kernel density estimation (KDE) of test dataset with respect to NLL and RMSE (averaged values of all six QoIs). The stars and circles represent the maximum and median points of each model, respectively.

3.2. Evaluation of regression performance

In this section, the regression performances of all selected models are presented using a test dataset that is not used in model training. Before going into details, DE-2 (which required the least training time among the DE models) is compared with GPR to highlight the efficiency of the DE models. Fig. 5 shows the results of kernel density estimation (KDE), which demonstrates the generalization performance of the models by visualizing the distributions of the test data in terms of NLL and RMSE (those of all six QoIs are averaged to be shown in this figure). For both criteria, the obvious superiority of DE-2 can be identified: most of the test data is concentrated in the lower error region in DE-2. More specifically, the KDE of NLL shows that the density peak of DE-2 represented by a star with long dashed line is located at NLL of -4.6 , while that of GPR is located at -3.1 . When it comes to RMSE, the peak of DE-2 is at RMSE of 0.003 while GPR is at 0.09 . The medians of the error metrics are also shown as circles with dotted lines. For both metrics, those of DE-2 are much lower than those of GPR, indicating that DE-2 performs better than GPR overall. The most interesting point here is that although DE-2 requires only 22% of the training time of GPR, it achieves superior regression accuracy.

Fig. 6 provides the comprehensive results of the regression performance. Fig. 6(a) shows the NLL results of all models with respect to the six aerodynamic QoIs, and their averaged NLL is also shown at the right end. Throughout all QoIs, GPR shows inferior regression accuracy than all other DE models. The results on NLL could be expected as each NN in the DE model is trained to minimize NLL. However, the results on RMSE in Fig. 6(b) are highly inspiring: they also achieve higher regression accuracy even in terms of RMSE. Considering that numerous engineers use RMSE to evaluate regression models, the fact that the average RMSE of DE models is less than half that of GPR is quite encouraging. Also, contrary to the claim that DE-5 would be sufficient in the work first proposed DE approach (Lakshminarayanan et al., 2017), DE-2 seems to be sufficient enough in this study, at least in terms of predictive accuracy: the difference in their values between all DE models is insignificant. However, the conventional belief is that the more models used in the ensemble, the more accurate the prediction will be due to the robustness that comes from averaging multiple predictions. The underlying reason for this counter-intuitive result (that is, insignificant differences in predictive accuracy as M increases) is thought to be the insufficient diversity within individual models due to the strategy adopted by DE: identical dataset and

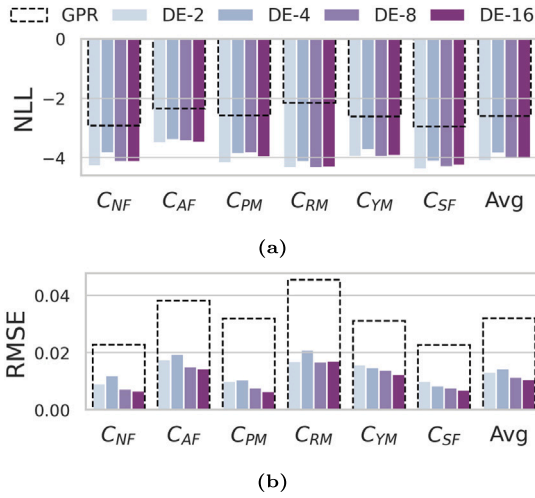


Fig. 6. Comparison of regression accuracy between GPR and all DE models: comprehensive results in terms of all aerodynamic QoIs. (a) NLL, (b) RMSE.

model architecture (Lakshminarayanan et al., 2017). However, note that blindly ensuring excessive diversity by using different datasets and model architectures should be done with caution, since it can degrade UQ performance (which will be shown in **Remark 2** of Section 3.4). In this regard, a trade-off study between the predictive accuracy and uncertainty quality as the diversity varies within individual NNs may be an interesting future work.

3.3. Evaluation of UQ performance

This section examines the quality of the predictive uncertainty, using AUCE and ENCE criteria for the quantitative investigation. For this purpose, reliability plots should be drawn first, using the test dataset split in Section 3.1 (dataset size of 980). Also, as in Algorithm 2, CI-based reliability plots require the set of CI candidates (P) and error-based reliability plots in Algorithm 3 need the number of bins (B). In this study, $P = \{0.1, 0.2, \dots, 0.9\}$ and $B = 20$ are chosen.

Fig. 7 shows the results of GPR, and it appears that GPR has a satisfactory uncertainty quality with respect to the error-based reliability plot (Fig. 7(b)), while the CI-based plot (Fig. 7(a)) shows relatively poor quality. In a CI-based plot, since the predicted CI (x-axis) is underestimated compared to the actual observed CI (y-axis), it can be inferred that GPR is trained to be “underconfident”: it is underconfident itself, so it overestimates its uncertainty.

The results of the DE models are then shown in Fig. 8. Note that unlike GPR in Fig. 7, only the results of output C_{SF} are visualized to highlight the differences between DE models: comprehensive results can be found in Fig. C.13 in Appendix C. For DE-2, the CI-based plot (Fig. 8(a)) shows a similar trend to that of GPR, while the error-based plot (Fig. 8(b)) shows slightly better quality. Meanwhile, a notable trend is observed along the increase of M : as it increases, the uncertainty quality with respect to both reliability plots apparently degrades. More specifically, both types of plots move upward away from the $y = x$ ideal line as M increases, indicating that DE models tend to become “underconfident”. Considering that DE-16 requires about 8 times as much training time as DE-2, it can be confirmed that using large M values for the ensemble does not necessarily lead to better results, but rather the opposite in terms of uncertainty quality. In this context, assuming that the performance of DE-5 will be between DE-4 and DE-8, it can be inferred that using $M = 5$ as suggested by Lakshminarayanan et al. (2017) does not guarantee sufficient UQ quality in this case. In fact, the insight behind this underconfident tendency when ensembling networks in classification tasks can be found in Rahaman et al.

(2021), while the corresponding tendency in regression has not been proven. Accordingly, in the next section, we provide the mathematical explanation for this underconfident tendency in regression tasks.

3.4. Theoretical derivation: underconfidence of DE in regression tasks

This section is for the mathematical derivation of why the ensemble of NNs becomes underconfident, as discovered in the previous section. For this purpose, the deviation from calibration (DC) score is introduced as in Rahaman et al. (2021); their work focused only on classification tasks, so their DC score consisted of the Brier score and the entropic term. Meanwhile, since our work focuses on the regression task, we adopted the different DC score consisting of the MSE and predictive variance. In this context, the following proposition and its proof can be considered as one of the contributions of this paper.

Proposition. When DC score is defined as follows,

$$DC(\mu, \sigma) \equiv (y - \mu)^2 - \sigma^2 \quad (11)$$

DC score of the ensemble becomes less than or equal to the averaged DC score of the individual NNs.

$$DC(\hat{\mu}, \hat{\sigma}) \leq \frac{1}{M} \sum_{i=1}^M DC(\mu_i, \sigma_i) \quad (12)$$

Proof. The averaged DC score of the individual NNs (right-hand side of the Eq. (12)) can be expressed as:

$$\begin{aligned} \frac{1}{M} \sum_{i=1}^M DC(\mu_i, \sigma_i) &= \frac{1}{M} \sum_{i=1}^M (y^2 - 2y\mu_i + \mu_i^2 - \sigma_i^2) \\ &= y^2 - 2y\hat{\mu} + \frac{1}{M} \sum_{i=1}^M \mu_i^2 - \frac{1}{M} \sum_{i=1}^M \sigma_i^2 \\ &= (y^2 - 2y\hat{\mu} + \hat{\mu}^2 - \hat{\sigma}^2) + \left(\frac{1}{M} \sum_{i=1}^M \mu_i^2 - \hat{\mu}^2 \right) + (\hat{\sigma}^2 - \frac{1}{M} \sum_{i=1}^M \sigma_i^2) \\ &= \underbrace{(y^2 - 2y\hat{\mu} + \hat{\mu}^2 - \hat{\sigma}^2)}_{=DC(\hat{\mu}, \hat{\sigma})} + 2 \underbrace{\left(\frac{1}{M} \sum_{i=1}^M \mu_i^2 - \hat{\mu}^2 \right)}_{=Var(\mu_i)} \quad (\because \text{Eq. (6)}) \end{aligned} \quad (13)$$

Hence,

$$DC(\hat{\mu}, \hat{\sigma}) = \frac{1}{M} \sum_{i=1}^M DC(\mu_i, \sigma_i) - \underbrace{2 \cdot Var(\mu_i)}_{\geq 0} \quad \square \quad (14)$$

Remark 1. The DC used in the above proposition indicates the degree of calibration. When DC equals 0, it means that the estimated uncertainty σ^2 exactly matches the MSE, $(y - \mu)^2$. If $DC < 0$, the uncertainty is overestimated compared to the MSE, which is an underconfident case. Therefore, the proposition that the DC score decreases after ensembling has mathematically explained the underconfidence of DE models observed in Section 3.3.

Remark 2. In Section 3.2, it was mentioned that introducing excessive diversity to individual NNs can lead to degraded UQ performance. This can be easily inferred by the term $Var(\mu_i)$ in Eq. (14): the more variance NNs have, the more underconfidence their ensemble shows.

4. DE models with STD calibration

The underconfidence tendency of DE models in regression tasks is observed and explained in the previous section. This section suggests the use of post-hoc STD calibration to mitigate this undesirable tendency and examines its effects.

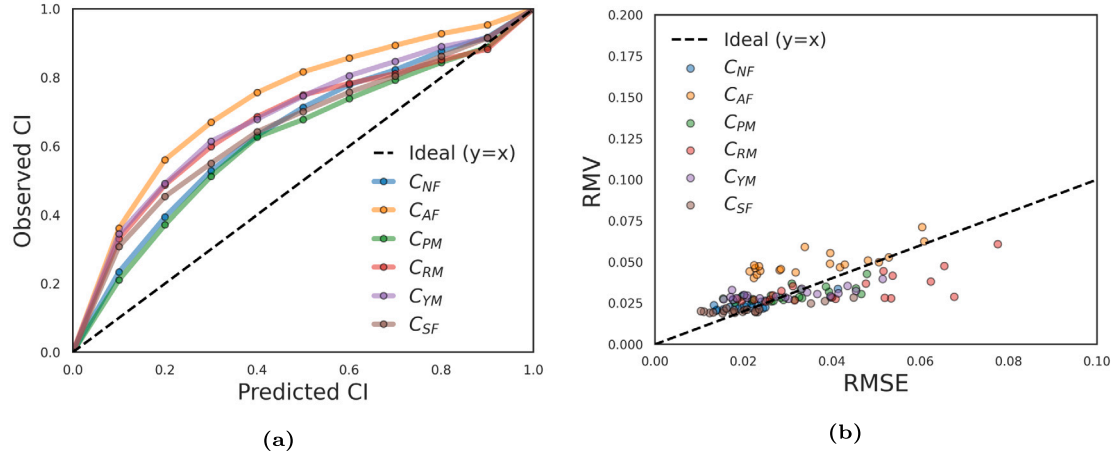


Fig. 7. Reliability plots of GPR: (a) CI-based reliability plot, (b) Error-based reliability plot.

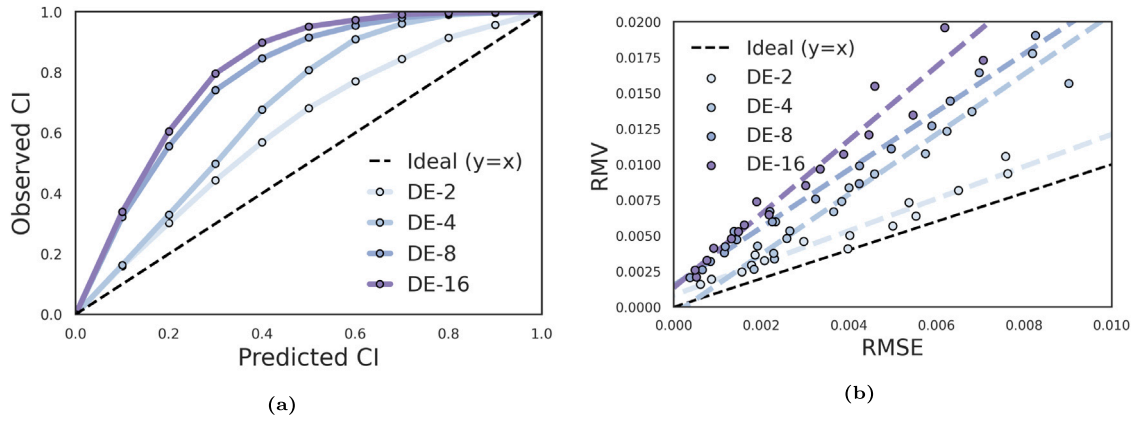


Fig. 8. Reliability plots of DE: for simplicity, only the C_{SF} results of different DE models are shown. (a) CI-based reliability plot, (b) Error-based reliability plot. In (b), to clearly show the decreasing tendency of UQ quality with increasing M , the linear regression model of the scatter points of each DE model is shown as a dashed line with the corresponding color.

4.1. STD calibration of DE models

The findings presented in Section 3.3 suggest that, despite the prevailing view that DE models are well-calibrated, this is not always the case, as illustrated in this straightforward multi-output regression task within an engineering domain. To address this issue, we propose using the STD calibration method on the trained DE models. This technique, as described in Algorithm 4, is straightforward and practical, as it requires only a single for-loop and leverages the existing models without additional training. This makes it a feasible option for our study, which focuses on the application of DE in engineering, where practicality is crucial.

Algorithm 4 first requires a set of candidates for scaling factors, S . Since the scaling factor of 1 corresponds to the case without calibration, the candidates s are set around 1. Accordingly, $s = 10^x$ are chosen as candidates, where x are 100 uniformly distributed points from -2 to 0.18 , so that the resulting range of scaling factors to explore is from 0.01 to 1.5 . Note that with s less than 1, underconfident models that overestimate the standard deviations (uncertainty) can be calibrated. Finally, the STD calibration is performed using validation dataset split in Section 3.1 (dataset size of 980) and the optimized scaling factors for each DE model with respect to each output (QoI) are summarized in Table 1. The STD calibration for all models is performed within 60 s, which is negligible compared to their training time.

Herein, the scaling factors for all six aerodynamic coefficients and their average value in each model are presented. The most notable point is that almost all s values are less than 1 and they decrease as

Table 1

Optimized scaling factors for STD calibration.

Methods	Optimized scaling factors						Avg
	C_{NF}	C_{AF}	C_{PM}	C_{RM}	C_{YM}	C_{SF}	
DE-2	0.549	1.061	0.608	1.009	0.824	0.578	0.771
DE-4	0.385	0.405	0.284	0.472	0.257	0.270	0.345
DE-8	0.147	0.270	0.133	0.270	0.155	0.140	0.186
DE-16	0.103	0.199	0.088	0.189	0.120	0.108	0.135

M increases: see the bold values in Table 1 to confirm their average trend. Taken together with the results from Section 3.3 that DE models overestimate their σ^2 (become underconfident) as M increases, one might expect optimized $s < 1$ to mitigate this underconfident tendency. And Fig. 9 proves that this actually happens: reliability plots of the DE models after STD calibration are drawn with the test dataset. Note that the validation dataset used during the STD calibration should not be reused in this process for generalization purposes. When compared to the previous plots in Fig. 8, the obvious improvement due to the calibration technique can be observed. See Appendix D for comprehensive results on the calibration effects with respect to all six QoIs.

Then, the quantitative effects of the calibration in terms of AUCE and ENCE will be analyzed, and from now on DE before and after calibration will be referred to as DE-bef and DE-aft, respectively. The AUCE and ENCE of the GPR will also be presented for the comparison, but please note that the GPR can be considered inherently STD-calibrated since its training algorithm already aims to minimize NLL as in the

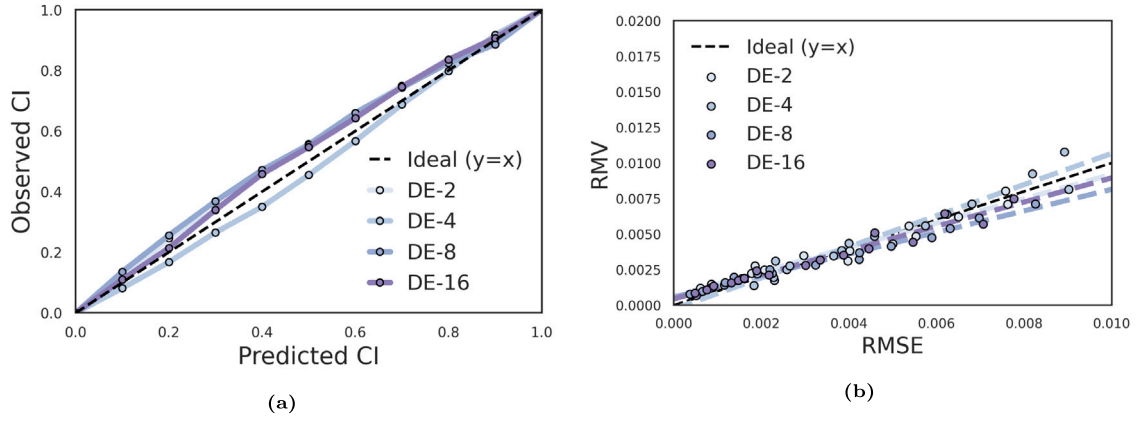


Fig. 9. Reliability plots of DE after STD calibration: (a) CI-based reliability plot, (b) Error-based reliability plot. The noticeable effects of STD calibration can be found when compared with the corresponding figure before STD calibration, Fig. 8.

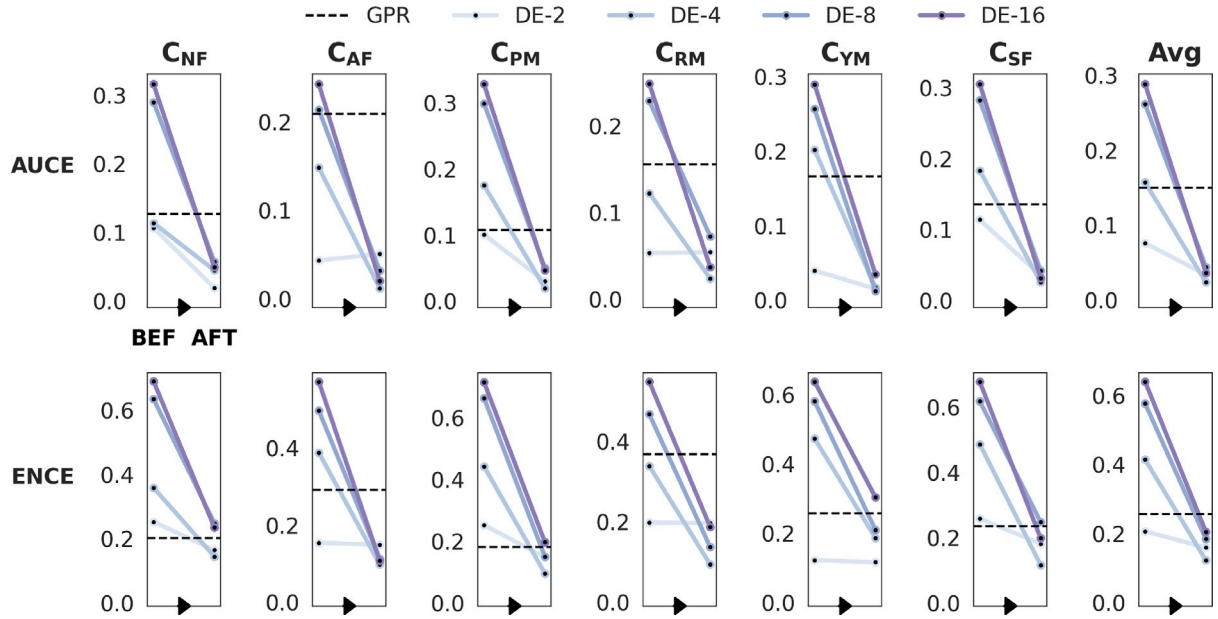


Fig. 10. AUC and ENCE of DE models before and after STD calibration. Those of GPR are also shown for comparison.

STD calibration process. This means that the GPR does not require additional STD calibration for a fair comparison with DE-aft because it can be seen as having already undergone STD calibration. Finally, the results are summarized in Fig. 10. It consists of the sub-figures, where the row indicates each UQ metric, the column indicates each QoI, and the x-axis in each sub-figure indicates whether the DE undergoes STD calibration (as explained, GPR metrics have a constant value along the x-axis regardless of the STD calibration). Before the calibration, the AUC (upper row) of GPR is between DE models: DE-2 is better than GPR, DE-4 is similar, and DE-8 and DE-16 are worse. However, the STD calibration completely changes this situation: AUC of all DE models for all aerodynamic QoIs decreases dramatically. For all outputs, DE-aft clearly outperforms GPR. The significant improvement of ENCE (lower row) due to the calibration of DE can also be verified. DE models show worse performance than GPR without calibration, but this gap narrows and even reverses, as seen in the rightmost subplot “Avg”, which shows the average performance of all outputs. At least in our study, DE-2 with STD calibration can be regarded as the best model, since DE models show negligible differences in predictive accuracy, UQ quality after calibration, while DE-2 requires the shortest training time (note that this does not mean that the lower M is better; careful consideration of M in each situation is a prerequisite, as in this study). In summary,

vanilla DE outperformed GPR in terms of training efficiency and regression accuracy, but not in terms of quality of estimated uncertainty. However, when used with a simple post-hoc STD calibration (which requires negligible additional post-processing time), DE demonstrated its strong potential as an alternative to GPR in terms of training time, prediction accuracy, and also UQ quality.

4.2. Effects of STD calibration on exploratory behavior in Bayesian optimization

Since the scaling factors are optimized to have values less than 1 during the STD calibration process (Table 1), it is obvious that the overall predictive uncertainty of DE models would decrease. To provide a more intuitive understanding of the practical implications of calibration, this section aims to briefly point out that applying STD calibration to DE can lead to different exploratory behavior during Bayesian optimization. Specifically, the importance of calibration is highlighted by comparing the next query candidates before and after STD calibration obtained in the first iteration of Bayesian optimization. Note that only the first iteration is implemented in this paper for the following two reasons. First, the goal of this section is simply to show the impact of the calibration from a practical point of view. Second, the

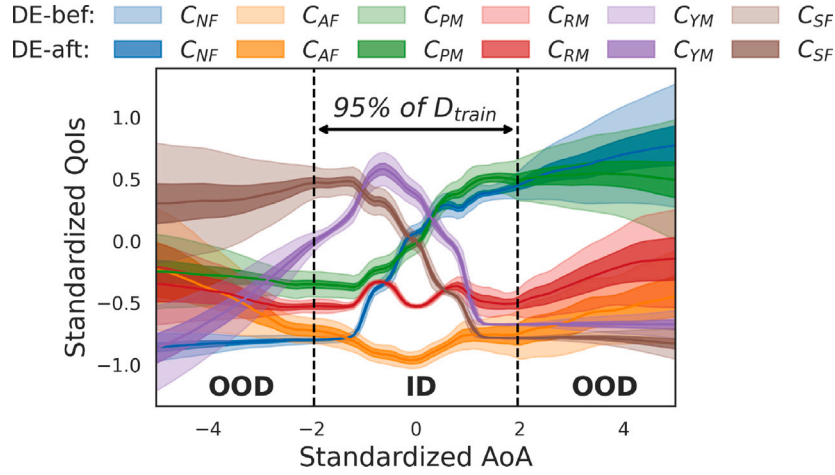


Fig. 11. CIs of 68% confidence level predicted by DE-16: comparison between before and after STD calibration.

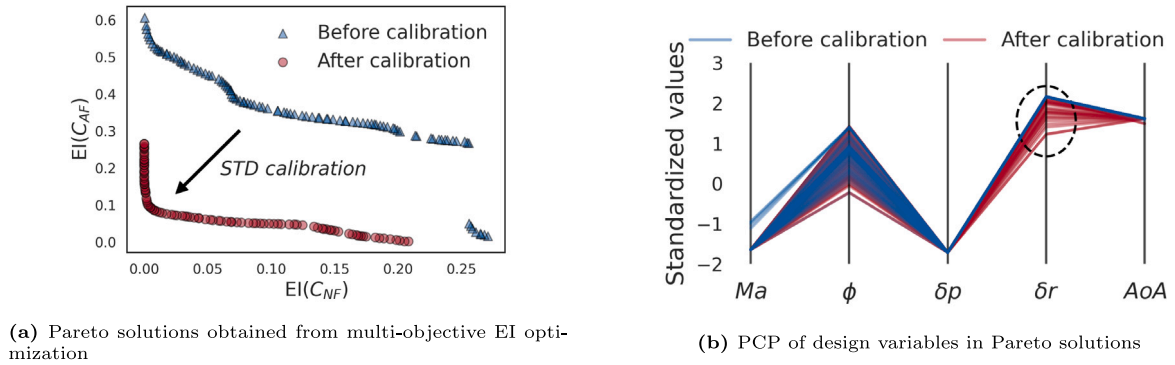


Fig. 12. Effects of STD calibration for DE models on Bayesian optimization results.

purpose of this section is not to claim that the final converged results of Bayesian optimization can be different depending on the calibration. Rather, it is to point out that the intended balance between exploitation and exploration may not be realized due to the miscalibrated uncertainty of the vanilla DE, which may affect the convergence behavior of the Bayesian optimization (by “intended balance” we mean the balance between exploration/exploitation in EI when the exact uncertainty is quantified).

Before moving on to Bayesian optimization, CIs of the 68% confidence level predicted by DE-16 model are shown in Fig. 11 to visually understand the impact of calibration. Only one input variable, AoA , is used for the illustration. And its value is standardized to distinguish between its ID (in-distribution) region and the OOD (out-of-distribution) region: in Fig. 11, the ID region is defined as the area containing 95% of the train data, while the OOD region is the remaining area. Overall, both results – those obtained before and after STD calibration – show diverging CIs in OOD and relatively narrow CIs in ID for all six QoIs. However, as expected from the scaling factors less than 1, the CIs from the DE-aft models become significantly narrower than those from the DE-bef models, indicating that these discrepancies will lead to differences in the subsequent Bayesian optimization process.

Then, the multi-objective Bayesian optimization problem is defined is adopted to practically investigate their effects on Bayesian optimization: maximization of both C_{NF} and C_{AF} within five varying input parameters (Ma , ϕ , δp , δr , and AoA). These optimizations, coupled with the expected improvement (EI) acquisition function, are performed separately for DE-bef and DE-aft models. The former searches for the maximum EI point where EI is calculated from the uncertainty quantified by the DE-bef model, while the latter does so using the uncertainty quantified by DE-aft. To find the Pareto solutions of $EI(C_{NF})$ and

$EI(C_{AF})$, the non-dominated sorting genetic algorithm-II (NSGA-II) in the Python package pymoo is utilized (Blank and Deb, 2020; Yang et al., 2022; Öztürk et al., 2006). Finally, the obtained Pareto solutions from the first iteration are shown in Fig. 12(a). Since the uncertainty estimated by DE-bef and DE-aft are different as shown in Fig. 11, the Pareto solutions of $EI(C_{NF})$ and $EI(C_{AF})$ are also different: EI values of both QoIs after calibration are much smaller than those before calibration.

In Bayesian optimization, however, the most valuable information to the user is not the EI value itself (Fig. 12(a)). More important are the values of the input variable sets (Fig. 12(b)) obtained from the EI Pareto solutions: they are the next query candidates, the main purpose of implementing Bayesian optimization. Additional experiments/simulations will be performed on these candidates, indicating that their selection has a significant impact on the convergence behavior of the iterative Bayesian optimization process. If unintended candidates are obtained due to inaccurate UQ and therefore inaccurate EI calculation, the exploratory behavior of Bayesian optimization can be much different from the intention of the user. That is, the intended balance between exploitation and exploration during Bayesian optimization may differ due to unintentionally overestimated/underestimated uncertainty. To inspect its unintended exploratory behavior more intuitively, the parallel coordinates plot (PCP) in Fig. 12(b) shows how the first query candidates in Bayesian optimization can vary due to the STD calibration in the DE model. This PCP has five vertical lines corresponding to each input variable, and the y-axis indicates their standardized values. Each red/blue line represents each point of the Pareto solutions in Fig. 12(a). Comparing them, large variations are found especially in the input variable δr : Bayesian optimization coupled with DE-bef discourages exploration of the variable δr (which was not intended by

Table 2
Comprehensive comparison between GPR and DE-2.

Metrics		GPR	DE-2	
			Before calibration	After calibration
Regression	NLL	−2.653 (−%)	−4.145 (↓ 56%)	−4.145 (↓ 56%)
	RMSE	0.029 (−%)	0.013 (↓ 55%)	0.013 (↓ 55%)
UQ	AUCE	0.150 (−%)	0.076 (↓ 49%)	0.034 (↓ 77%)
	ENCE	0.256 (−%)	0.206 (↓ 20%)	0.159 (↓ 38%)
Training time [s]		39081 (−%)	8640 (↓ 78%)	8640+30 (↓ 78%)

the user), while DE-aft encourages exploration within δr (which was the original intention). In conclusion, whether the DE is calibrated by STD calibration or not can result in exploration characteristics during Bayesian optimization that differ from the user's intent, which shows the impact of the calibration on DE from a practical point of view.

5. Conclusion

This study comprehensively investigated the state-of-the-art approximate Bayesian inference approach, DE. It is applied to the multi-output regression task, which is the most common task in the engineering fields: a simple test case is adopted where aerodynamic QoIs of the specific missile configuration are predicted under varying flow conditions. DE models with different numbers of NNs are trained and then examined in the following order. First, their regression performance and the quality of estimated uncertainty are scrutinized while being compared with GPR. Then, a simple post-hoc STD calibration method is proposed to be applied to miscalibrated DE models. Finally, the effectiveness of the calibration on DE is highlighted by the improvement of two UQ quality criteria and the different exploratory behavior in Bayesian optimization before and after calibration. The key findings of our study can be summarized as follows:

1. The effect of the number of NNs used in ensemble, M , is comprehensively investigated in the simple multi-output regression task. For regression accuracy, DE models show superior performance to GPR in terms of RMSE and NLL, while showing indistinguishable differences among themselves. For UQ quality, however, they show the obvious trend towards underconfidence as M increases, both in terms of AUCE and ENCE criteria. The mathematical proof of why DE tends to be miscalibrated in regression tasks is also derived.
2. The post-hoc STD calibration method, which simply modifies the estimated uncertainty from DE, is proposed to be applied to miscalibrated DE models. Finally, the reliability of the UQ performance after calibration is dramatically improved for both AUCE and ENCE, also surpassing that of GPR.
3. The impact of the calibration approach on the exploratory behavior in Bayesian optimization is examined. Finally, whether or not the DE is calibrated via STD calibration can result in completely different exploration characteristics when extended to Bayesian optimization, which cautions against blindly applying vanilla DE models to Bayesian optimization in regression tasks.
4. We have demonstrated that by applying a simple post-hoc STD calibration technique that requires negligible additional post-processing time, DE models can have enormous potential compared to GPR, which is the most commonly used regression model for UQ in engineering. These results are summarized in Table 2, where the DE-2 model after STD calibration outperforms GPR in terms of regression performance (−56% NLL & −55% RMSE), reliability of UQ (−77% AUCE & −38% ENCE), and training efficiency (−78% training time).

The presented DE framework has great promise in two engineering applications. First, DE with STD calibration has the potential to replace the most common regression model, GPR, owing to its following advantages: more scalable to large datasets, higher regression accuracy, and last but not least, more reliable uncertainty estimation. Second, DE with STD calibration can be leveraged in Bayesian optimization by ensuring a reliable balance between exploitation and exploration due to its trustworthy UQ performance. Although the application of this framework has been demystified using the simple multi-output regression task, it can be easily applied and extended to high-dimensional input/output problems since it is based on the deep neural network structures and no special assumptions have been made for this specific problem. For future work, a more practical investigation of DE models will be conducted, such as their scalability to high-dimensional engineering regression problems, not as simple regression task with 6 outputs in this study. Also, since the DE algorithm by its nature requires multiple network training, the way to reduce its longer training time than the conventional NNs can also be a future study. Finally, since our purpose was not to claim that the final converged results of Bayesian optimization can be different depending on the calibration, the extension of DE to the whole Bayesian optimization was outside the focus of our paper. In this respect, a comprehensive comparison of the convergence behavior between DE-bef and DE-aft over entire iterations and their final converged results can be invaluable future work.

CRedit authorship contribution statement

Sunwoong Yang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Kwanjung Yee:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by Grant UE191109CD from Agency for Defense Development and Defense Acquisition Program Administration & Hyundai Motor Group. Also, the authors especially thank Shinkyu Jeong and Seungmin Yoo at Kyunghee University for providing the training dataset of the current study.

Appendix A. Controversial issues on MC-dropout

Osband (2016) pointed out that what MC-dropout (MCD) estimates is a risk, not an uncertainty, and also emphasized the pitfalls of MCD when used as a naive tool for estimating uncertainty. Moreover, its algorithm does not perform adequately even in very simple examples (Osband et al., 2016; Pearce et al., 2018), and its posterior samples are often too spiky to provide a reliable predictive uncertainty trend (Gal and Ghahramani, 2016; Gal et al., 2016; Osband et al., 2016; Riquelme et al., 2018; Zhang et al., 2019), which makes it unattractive to be exploited for Bayesian optimization.

Appendix B. Hyperparameter tuning results in Section 3.1

B.1. Results of DE models

The results of the hyperparameter tuning for DE models performed in Section 3.1 are shown in Table B.3. Three hyperparameters are used: the number of hidden layers ($N_{layer} \in \{3, 5, 7\}$), the number of nodes in each hidden layer ($N_{node} \in \{32, 64, 128\}$), and the size of the mini-batch ($N_{batch} \in \{512, 1024, 2048\}$). The corresponding regression performance in terms of NLL and RMSE for all hyperparameter combinations is shown (total training time of 6944 s). Since $N_{layer} = 7, N_{node} = 128, N_{batch} = 512$ shows the best RMSE performance, it is selected as the best hyperparameter combination.

B.2. Results of GPR models

The results of the hyperparameter tuning for GPR models performed in Section 3.1 are shown in Table B.4. For conventional single-output

Table B.3

Results of hyperparameter tuning: several structures of probabilistic NN used in the DE model are tested.

N_{layer}	N_{node}	N_{batch}	NLL	RMSE	Time [s]
3	32	512	-2.50	0.125	183
		1024	-2.22	0.200	174
		2048	-2.13	0.183	183
	64	512	-2.98	0.076	202
		1024	-2.69	0.087	179
		2048	-2.52	0.100	213
	128	512	-3.37	0.036	256
		1024	-3.34	0.040	257
		2048	-2.55	0.052	271
	32	512	-2.62	0.137	187
		1024	-2.23	0.173	214
		2048	-2.25	0.137	202
5	64	512	-3.21	0.039	236
		1024	-2.79	0.055	210
		2048	-2.01	0.085	255
	128	512	-3.83	0.017	342
		1024	-3.53	0.019	327
		2048	-3.16	0.035	349
	32	512	-2.49	0.088	212
		1024	-2.36	0.119	224
		2048	-2.17	0.104	244
7	64	512	-3.38	0.031	286
		1024	-3.05	0.042	247
		2048	-2.45	0.062	262
	128	512	-3.82	0.016	428
		1024	-3.56	0.024	399
		2048	-3.13	0.035	402

Table B.4

Results of hyperparameter tuning: several GPR models are tested.

Kernels	NLL	RMSE	Time [s]
Matérn 5/2	-2.653	0.029	39 081
Radial basis function (SOGPR)	-1.657	0.039	64 250
Radial basis function (MOGPR)	-1.4236	0.044	8136
Rational quadratic	-0.747	0.061	66 363
Dot-product	0.440	0.547	27 663

GPR (SOGPR), Matérn 5/2, radial basis function, rational quadratic, and dot-product kernels are explored. Additionally, multi-output GPR (MOGPR) with radial basis function is also tested. The corresponding regression performance in terms of NLL and RMSE of all GPR models are summarized in Table B.4 (total training time is 205 493 s, which is significantly longer than DE in Appendix B.1). MOGPR requires the least training time, but single-output GPR with Matérn 5/2 is selected since it shows the best performance with respect to NLL and RMSE.

Appendix C. Additional results in Section 3.3

Fig. 8 in Section 3.3 shows the reliability plots with respect to only one QoI, C_{SF} . In this section, more comprehensive results are provided in Fig. C.13. For each vanilla DE model, all six QoIs are shown with different colors. Again, the trend of underconfidence as M increases can be seen from DE-2 to DE-16.

Appendix D. Additional results in Section 4.1

In Appendix C, the reliability plots of DE models before STD calibration (vanilla DE models) are shown; this section shows the results after STD calibration (see Fig. D.14). It is shown that all DE models become well-calibrated after calibration, even for the DE-16 model, which was the most miscalibrated DE model.

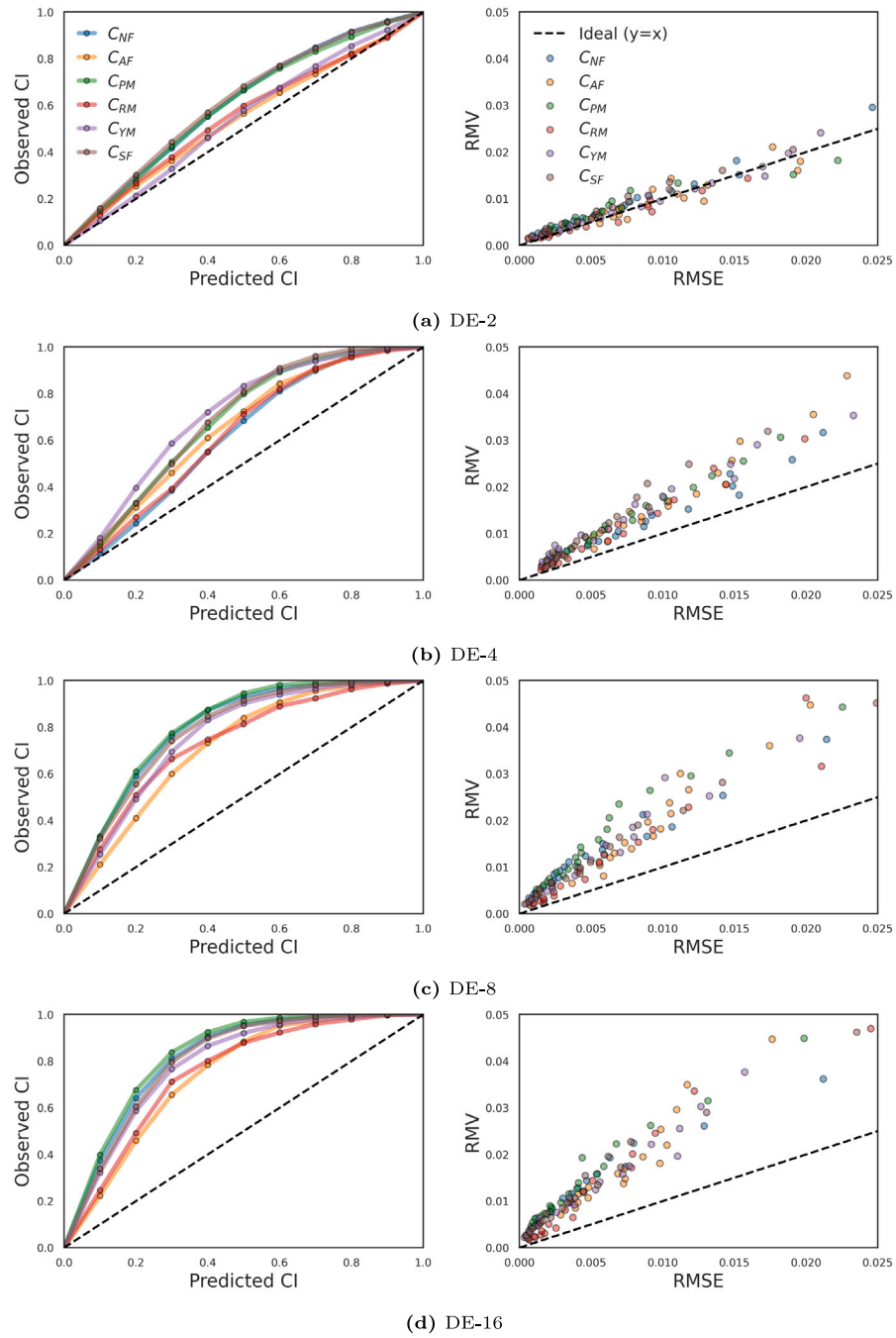


Fig. C.13. Reliability plots of vanilla DE models: (left) CI-based reliability plots, (right) error-based reliability plots.

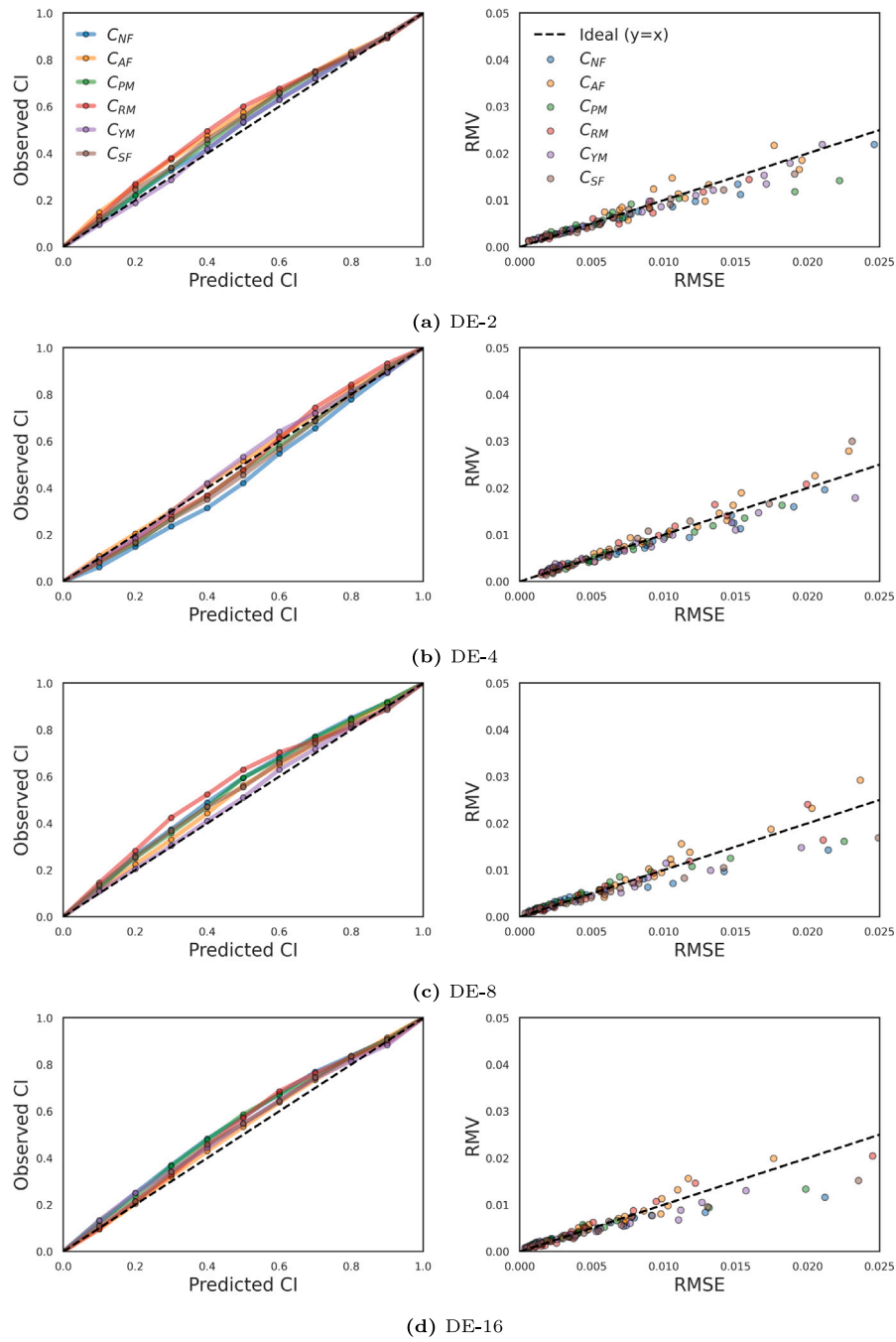


Fig. D.14. Reliability plots of DE models after STD calibration: (left) CI-based reliability plots, (right) error-based reliability plots.

References

- Allen, J.M., 2005. Aerodynamics of an Axisymmetric Missile Concept Having Cruciform Strakes and In-Line Tail Fins from Mach 0.60 to 4.63. Technical Report.
- Alvarez, M.A., Rosasco, L., Lawrence, N.D., et al., 2012. Kernels for vector-valued functions: A review. *Found. Trends® Mach. Learn.* 4 (3), 195–266.
- Ashukha, A., Lyzhov, A., Molchanov, D., Vetrov, D., 2020. Pitfalls of in-domain uncertainty estimation and ensembling in deep learning. *arXiv preprint arXiv:2002.06470*.
- Barron, A.R., 1993. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Trans. Inf. Theory* 39 (3), 930–945.
- de Becdelievre, J., Kroo, I., 2021. A Bayesian approach to collaborative optimization with application to tailless aircraft range maximization. In: *AIAA AVIATION 2021 FORUM*. p. 3063.
- Blake, W.B., 1998. Missile Datcom: User's Manual-1997 FORTRAN 90 Revision. Technical Report, Air Force Research Lab Wright-Patterson AFB OH Air Vehicles Directorate.
- Blank, J., Deb, K., 2020. Pymoo: Multi-objective optimization in python. *IEEE Access* 8, 89497–89509.
- Chae, S., Yee, K., Yang, C., Aoyama, T., Jeong, S., Obayashi, S., 2010. Helicopter rotor shape optimization for the improvement of aeroacoustic performance in hover. *J. Aircr.* 47 (5), 1770–1783.
- Cheng, C.-A., Boots, B., 2017. Variational inference for Gaussian process models with linear complexity. *Adv. Neural Inf. Process. Syst.* 30.
- Deruyttere, T., Milewski, V., Moens, M.-F., 2021. Giving commands to a self-driving car: How to deal with uncertain situations? *Eng. Appl. Artif. Intell.* 103, 104257.
- Duchi, J., Hazan, E., Singer, Y., 2011. Adaptive subgradient methods for online learning and stochastic optimization. *J. Mach. Learn. Res.* 12 (7).
- Egele, R., Maulik, R., Raghavan, K., Lusch, B., Guyon, I., Balaprakash, P., 2022. Autodequ: Automated deep ensemble with uncertainty quantification. In: *2022 26th International Conference on Pattern Recognition. ICPR, IEEE*, pp. 1908–1914.
- Fernández, J., Chiachío, J., Chiachío, M., Barros, J., Corbetta, M., 2023. Physics-guided Bayesian neural networks by ABC-SS: Application to reinforced concrete columns. *Eng. Appl. Artif. Intell.* 119, 105790.
- Fernández, J., Chiachío, M., Chiachío, J., Muñoz, R., Herrera, F., 2022. Uncertainty quantification in neural networks by approximate Bayesian computation: Application to fatigue in composite materials. *Eng. Appl. Artif. Intell.* 107, 104511.
- Folgor, L.L., Baltatzis, V., Desai, S., Devaraj, A., Ellis, S., Manzanera, O.E.M., Nair, A., Qiu, H., Schnabel, J., Glocker, B., 2021. Is MC dropout Bayesian? *arXiv preprint arXiv:2110.04286*.
- Fort, S., Hu, H., Lakshminarayanan, B., 2019. Deep ensembles: A loss landscape perspective. *arXiv preprint arXiv:1912.02757*.
- Gal, Y., Ghahramani, Z., 2016. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: *International Conference on Machine Learning. PMLR*, pp. 1050–1059.
- Gal, Y., et al., 2016. Uncertainty in Deep Learning. (Ph.D. thesis). University of Cambridge.
- Gneiting, T., Raftery, A.E., 2007. Strictly proper scoring rules, prediction, and estimation. *J. Am. Stat. Assoc.* 102 (477), 359–378.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. *Deep Learning*. MIT Press.
- Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks. In: *International Conference on Machine Learning. PMLR*, pp. 1321–1330.
- Gustafsson, F.K., Danelljan, M., Schon, T.B., 2020. Evaluating scalable Bayesian deep learning methods for robust computer vision. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. pp. 318–319.
- Hastie, T., Tibshirani, R., Friedman, J.H., Friedman, J.H., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Vol. 2. Springer.
- Hong, Y., Lee, D., Yang, S., Kook, H., Yee, K., 2023. Exploration of stacked rotor designs for aerodynamics in hover. *Aerosp. Sci. Technol.* 141, 108557.
- Hornik, K., Stinchcombe, M., White, H., 1989. Multilayer feedforward networks are universal approximators. *Neural Netw.* 2 (5), 359–366.
- Hron, J., Matthews, A.G.d.G., Ghahramani, Z., 2017. Variational Gaussian dropout is not Bayesian. *arXiv preprint arXiv:1711.02989*.
- Hron, J., Matthews, A., Ghahramani, Z., 2018. Variational Bayesian dropout: Pitfalls and fixes. In: *International Conference on Machine Learning. PMLR*, pp. 2019–2028.
- Hu, S., Pezzotti, N., Welling, M., 2021. Learning to predict error for mri reconstruction. In: *International Conference on Medical Image Computing and Computer-Assisted Intervention. Springer*, pp. 604–613.
- Ilg, E., Cicek, O., Galesso, S., Klein, A., Makansi, O., Hutter, F., Brox, T., 2018. Uncertainty estimates and multi-hypotheses networks for optical flow. In: *Proceedings of the European Conference on Computer Vision. ECCV*, pp. 652–667.
- Jeong, S., Murayama, M., Yamamoto, K., 2005. Efficient optimization design method using Kriging model. *J. Aircraft* 42 (2), 413–420.
- Jones, D.R., Schonlau, M., Welch, W.J., 1998. Efficient global optimization of expensive black-box functions. *J. Global Optim.* 13 (4), 455–492.
- Kanazaki, M., Tanaka, K., Jeong, S., Yamamoto, K., 2007. Multi-objective aerodynamic exploration of elements' setting for high-lift airfoil using Kriging model. *J. Aircr.* 44 (3), 858–864.
- Kang, Y.-E., Yang, S., Yee, K., 2022. Physics-aware reduced-order modeling of transonic flow via β -variational autoencoder. *Phys. Fluids* 34 (7), 076103.
- Kendall, A., Gal, Y., 2017. What uncertainties do we need in Bayesian deep learning for computer vision? In: *Advances in Neural Information Processing Systems*. vol. 30.
- Keßler, T., Kunde, C., McBride, K., Mertens, N., Michaels, D., Sundmacher, K., Kienle, A., 2019. Global optimization of distillation columns using explicit and implicit surrogate models. *Chem. Eng. Sci.* 197, 235–245.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Kuleshov, V., Fenner, N., Ermon, S., 2018. Accurate uncertainties for deep learning using calibrated regression. In: *International Conference on Machine Learning. PMLR*, pp. 2796–2804.
- Lakshminarayanan, B., Pritzel, A., Blundell, C., 2017. Simple and scalable predictive uncertainty estimation using deep ensembles. In: *Advances in Neural Information Processing Systems*, vol. 30.
- Laves, M.-H., Ihler, S., Fast, J.F., Kahrs, L.A., Ortmaier, T., 2021. Recalibration of aleatoric and epistemic regression uncertainty in medical imaging. *arXiv preprint arXiv:2104.12376*.
- Levi, D., Gispán, L., Giladi, N., Fetaya, E., 2022. Evaluating and calibrating uncertainty prediction in regression tasks. *Sensors* 22 (15), 5540.
- Lin, Q., Hu, J., Zhang, L., Jin, P., Cheng, Y., Zhou, Q., 2022. Gradient-enhanced multi-output gaussian process model for simulation-based engineering design. *AIAA J.* 60 (1), 76–91.
- Lin, Q., Hu, J., Zhou, Q., Cheng, Y., Hu, Z., Couckuyt, I., Dhaene, T., 2021. Multi-output Gaussian process prediction for computationally expensive problems with multiple levels of fidelity. *Knowl.-Based Syst.* 227, 107151.
- Limans, J., van der Laak, J., Litjens, G., 2020. Efficient out-of-distribution detection in digital pathology using multi-head convolutional neural networks. In: *MIDL*. pp. 465–478.
- Maas, A.L., Hannun, A.Y., Ng, A.Y., et al., 2013. Rectifier nonlinearities improve neural network acoustic models. In: *Proc. Icml*, Vol. 30, no. 1. Atlanta, Georgia, USA, p. 3.
- MacKay, D.J., 1992. Information-based objective functions for active data selection. *Neural Comput.* 4 (4), 590–604.
- MacKay, D.J., 1995. Probable networks and plausible predictions—a review of practical Bayesian methods for supervised neural networks. *Network: Comput. Neural Syst.* 6 (3), 469.
- Maulik, R., Egele, R., Raghavan, K., Balaprakash, P., 2023. Quantifying uncertainty for deep learning based forecasting and flow-reconstruction using neural architecture search ensembles. *arXiv preprint arXiv:2302.09748*.
- Meng, X., Babae, H., Karniadakis, G.E., 2021. Multi-fidelity Bayesian neural networks: Algorithms and applications. *J. Comput. Phys.* 438, 110361.
- Muñoz, J., López, B., Quevedo, F., Garrido, S., Monje, C.A., Moreno, L.E., 2023. Gaussian processes and fast marching square based informative path planning. *Eng. Appl. Artif. Intell.* 121, 106054.
- Naeini, M.P., Cooper, G., Hauskrecht, M., 2015. Obtaining well calibrated probabilities using Bayesian binning. In: *Twenty-Ninth AAAI Conference on Artificial Intelligence*.
- Namura, N., Obayashi, S., Jeong, S., 2016. Efficient global optimization of vortex generators on a supercritical infinite wing. *J. Aircr.* 53 (6), 1670–1679.
- Nikolopoulos, S., Kalogeris, I., Papadopoulos, V., 2022. Non-intrusive surrogate modeling for parametrized time-dependent partial differential equations using convolutional autoencoders. *Eng. Appl. Artif. Intell.* 109, 104652.
- Osband, I., 2016. Risk versus uncertainty in deep learning: Bayes, bootstrap and the dangers of dropout. In: *NIPS Workshop on Bayesian Deep Learning*, Vol. 192.
- Osband, I., Blundell, C., Pritzel, A., Van Roy, B., 2016. Deep exploration via bootstrapped DQN. In: *Advances in Neural Information Processing Systems*, vol. 29.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., Snoek, J., 2019. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In: *Advances in Neural Information Processing Systems*, vol. 32.
- Öztürk, N., Yıldız, A.R., Kaya, N., Öztürk, F., 2006. Neuro-genetic design optimization framework to support the integrated robust design optimization process in CE. *Concurr. Eng.* 14 (1), 5–16.
- Park, K., Jung, J., Jeong, S., 2022. Multi-objective shape optimization of airfoils for mars exploration aircraft propellers. *Int. J. Aeronaut. Space Sci.* 1–15.
- Pawar, S., San, O., Vedula, P., Rasheed, A., Kvamsdal, T., 2022. Multi-fidelity information fusion with concatenated neural networks. *Sci. Rep.* 12 (1), 5900.
- Pearce, T., Anastassacos, N., Zaki, M., Neely, A., 2018. Bayesian inference with anchored ensembles of neural networks, and application to exploration in reinforcement learning. *arXiv preprint arXiv:1805.11324*.
- Phan, B., Salay, R., Czarniecki, K., Abdelzad, V., Denouden, T., Vernekar, S., 2018. Calibrating uncertainties in object localization task. *arXiv preprint arXiv:1811.11210*.
- Pocevičūtė, M., Eilertsen, G., Jarkman, S., Lundström, C., 2022. Generalisation effects of predictive uncertainty estimation in deep learning for digital pathology. *Sci. Rep.* 12 (1), 1–15.
- Quirante, N., Caballero, J.A., 2016. Optimization of a sour water stripping plant using surrogate models. In: *Computer Aided Chemical Engineering*, Vol. 38. Elsevier, pp. 31–36.

- Quirante, N., Javaloyes-Antón, J., Caballero, J.A., 2018. Hybrid simulation-equation based synthesis of chemical processes. *Chem. Eng. Res. Des.* 132, 766–784.
- Rahaman, R., et al., 2021. Uncertainty quantification and deep ensembles. *Adv. Neural Inf. Process. Syst.* 34, 20063–20075.
- Rhode, S., 2020. Non-stationary Gaussian process regression applied in validation of vehicle dynamics models. *Eng. Appl. Artif. Intell.* 93, 103716.
- Riquelme, C., Tucker, G., Snoek, J., 2018. Deep Bayesian bandits showdown: An empirical comparison of Bayesian deep networks for thompson sampling. *arXiv preprint arXiv:1802.09127*.
- Rumelhart, D.E., Hinton, G.E., Williams, R.J., 1986. Learning representations by back-propagating errors. *nature* 323 (6088), 533–536.
- Scalia, G., Grambow, C.A., Pernici, B., Li, Y.-P., Green, W.H., 2020. Evaluating scalable uncertainty estimation methods for deep learning-based molecular property prediction. *J. Chem. Inf. Model.* 60 (6), 2697–2717.
- Shimoyama, K., Sato, K., Jeong, S., Obayashi, S., 2013. Updating Kriging surrogate models based on the hypervolume indicator in multi-objective optimization. *J. Mech. Des.* 135 (9), 094503.
- Shin, S., Lee, Y., Kim, M., Park, J., Lee, S., Min, K., 2020. Deep neural network model with Bayesian hyperparameter optimization for prediction of NOx at transient conditions in a diesel engine. *Eng. Appl. Artif. Intell.* 94, 103761.
- Snoek, J., Larochelle, H., Adams, R.P., 2012. Practical Bayesian optimization of machine learning algorithms. In: *Advances in Neural Information Processing Systems*, vol. 25.
- Solopchuk, O., Zénon, A., 2021. Active sensing with artificial neural networks. *Neural Netw.* 143, 751–758.
- Sugimura, K., Jeong, S., Obayashi, S., Kimura, T., 2009. Kriging-model-based multi-objective robust optimization and trade-off rule mining of a centrifugal fan with dimensional uncertainty. *J. Comput. Sci. Technol.* 3 (1), 196–211.
- Tieleman, T., Hinton, G., et al., 2012. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Netw. Mach. Learn.* 4 (2), 26–31.
- VanDerHorn, E., Mahadevan, S., 2021. Digital twin: Generalization, characterization and implementation. *Decis. Support Syst.* 145, 113524.
- Wang, B., Chen, T., 2015. Gaussian process regression with multiple response variables. *Chemometr. Intell. Lab. Syst.* 142, 159–165.
- Wang, H., van Stein, B., Emmerich, M., Bäck, T., 2017. Time complexity reduction in efficient global optimization using cluster Kriging. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. pp. 889–896.
- Wang, B., Wang, W., Qiao, Z., Meng, G., Mao, Z., 2022. Dynamic selective Gaussian process regression for forecasting temperature of molten steel in ladle furnace. *Eng. Appl. Artif. Intell.* 112, 104892.
- Williams, C.K., Rasmussen, C.E., 2006. *Gaussian Processes for Machine Learning*, Vol. 2, no. 3. MIT press Cambridge, MA.
- Wilson, A.G., Izmailov, P., 2020. Bayesian deep learning and a probabilistic perspective of generalization. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 4697–4708.
- Wu, X., Gales, M., 2021. Should ensemble members be calibrated? *arXiv preprint arXiv:2101.05397*.
- Yang, H., Hong, S.H., ZhG, R., Wang, Y., 2020. Surrogate-based optimization with adaptive sampling for microfluidic concentration gradient generator design. *RSC Adv.* 10 (23), 13799–13814.
- Yang, S., Kim, H., Hong, Y., Yee, K., Romit, M., Kang, N., 2024. Data-driven physics-informed neural networks: a digital twin perspective. *arXiv preprint arXiv:2401.08667*.
- Yang, S., Lee, S., Yee, K., 2022. Inverse design optimization framework via a two-step deep learning approach: Application to a wind turbine airfoil. *Eng. Comput.* 1–17.
- Yang, S., Yee, K., 2022a. Comment on “novel approach for selecting low-fidelity scale factor in multifidelity metamodeling”. *AIAA J.* 60 (4), 2713–2715.
- Yang, S., Yee, K., 2022b. Design rule extraction using multi-fidelity surrogate model for unmanned combat aerial vehicles. *J. Aircr.* 1–15.
- Yang, S., Yee, K., 2023. Uncertainty quantification via deep ensembles in missile performance prediction. In: *AIAA SCITECH 2023 Forum*. p. 2684.
- Yıldız, B.S., 2020. Slime mould algorithm and Kriging surrogate model-based approach for enhanced crashworthiness of electric vehicles. *Int. J. Veh. Des.* 83 (1), 54–68.
- Yıldız, B.S., 2022. Marine predators algorithm and multi-verse optimisation algorithm for optimal battery case design of electric vehicles. *Int. J. Veh. Des.* 88 (1), 1–11.
- Zadrozny, B., Elkan, C., 2001. Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. In: *ICML*, Vol. 1. Citeseer, pp. 609–616.
- Zadrozny, B., Elkan, C., 2002. Transforming classifier scores into accurate multiclass probability estimates. In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 694–699.
- Zhang, H., Chen, W.W., Iyer, A., Apley, D.W., Chen, W., 2022. Uncertainty-aware mixed-variable machine learning for materials design. *Sci. Rep.* 12 (1), 1–13.
- Zhang, D., Lu, L., Guo, L., Karniadakis, G.E., 2019. Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems. *J. Comput. Phys.* 397, 108850.
- Zhong, W., Qiao, C., Peng, X., Li, Z., Fan, C., Qian, F., 2019. Operation optimization of hydrocracking process based on Kriging surrogate model. *Control Eng. Pract.* 85, 34–40.